



UNIVERSIDADE DE FORTALEZA
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

MATHEUS NAJAL CRUZ

**MODELAGEM E SIMULAÇÃO DE CORPOS E TECIDOS
DEFORMÁVEIS EM MOTORES DE JOGOS**

FORTALEZA – CEARÁ

2025

MATHEUS NAJAL CRUZ

MODELAGEM E SIMULAÇÃO DE CORPOS E TECIDOS DEFORMÁVEIS EM MOTORES DE JOGOS

Trabalho de Conclusão de Curso apresentado ao Curso de Graduação em Ciência da Computação do Centro de Ciências Tecnológicas da Universidade de Fortaleza, como requisito parcial à obtenção do grau de bacharel em Ciência da Computação.

Orientadora: Maria Andréia Formico Rodrigues, PhD

FORTALEZA – CEARÁ

2025

Ficha catalográfica da obra elaborada pelo autor através do programa de geração automática da Biblioteca Central da Universidade de Fortaleza

Cruz, Matheus Najal.

MODELAGEM E SIMULAÇÃO DE CORPOS E TECIDOS DEFORMÁVEIS EM
MOTORES DE JOGOS / Matheus Najal Cruz. - 2025
59 f.

Trabalho de Conclusão de Curso (Graduação) - Universidade
de Fortaleza. Curso de Ciência Da Computação, Fortaleza, 2025.
Orientação: Prof. Dra. Maria Andréia Formico Rodrigues.

1. Corpos e Tecidos Deformáveis. 2. Simulação Física. 3.
Motores de Jogos. 4. Aplicações Gráficas Interativas. I.
Rodrigues, Prof. Dra. Maria Andréia Formico . II. Título.



Unifor

Graduação em
Ciência da Computação

MODELAGEM E SIMULAÇÃO DE CORPOS E TECIDOS DEFORMÁVEIS EM MOTORES DE JOGOS

MATHEUS NAJAL CRUZ

PARECER: **APROVADO**

Data: 04 / 12 / 2025

BANCA EXAMINADORA:



Documento assinado digitalmente
JONAS DE ARAUJO LUZ JUNIOR
Data: 05/12/2025 09:27:15-0300
Verifique em <https://validar.iti.gov.br>

JONAS DE ARAÚJO LUZ JUNIOR, Msc.



Documento assinado digitalmente
HERBERT LIMA SANTOS DA ROCHA
Data: 05/12/2025 23:16:30-0300
Verifique em <https://validar.iti.gov.br>

HERBERT LIMA SANTOS DA ROCHA, MSc.



Documento assinado digitalmente
MARIA ANDREIA FORMICO RODRIGUES
Data: 05/12/2025 01:14:43-0300
Verifique em <https://validar.iti.gov.br>

MARIA ANDRÉIA FORMICO RODRIGUES, PhD.
Orientadora

Prof. Ma. Liádina Camargo Lima
Coordenadora do Curso de Ciência da Computação

AGRADECIMENTOS

À minha mãe, fonte inesgotável de amor e força, dedico a mais profunda gratidão. Por ter me concedido o dom da vida e por ser meu porto seguro nos momentos de tempestade, especialmente durante as crises de ansiedade e as incertezas que esta jornada impôs. Seu carinho, sua dedicação e os sacrifícios silenciosos foram o alicerce sobre o qual construí este sonho.

Ao meu pai, meu exemplo de perseverança e confiança. Agradeço por seu suporte inabalável ao longo de toda a graduação e por jamais duvidar da minha capacidade. Sua fé em meu potencial foi o combustível que me impulsionou a seguir em frente, mesmo diante dos maiores desafios.

À minha orientadora, Professora Maria Andréia Formico Rodrigues, minha eterna gratidão. Seus ensinamentos transcendem o conteúdo acadêmico; foram lições para a vida profissional e pessoal. Agradeço imensamente por sua paciência, pelas correções precisas e por me apresentar à área de estudo pela qual hoje sou apaixonado. Este trabalho é fruto direto de sua mentoria e inspiração.

“he had been ready to give his existence a thousand times over for an idea, a hope, even a fantasy. Existence alone had never been enough for him; he had always wanted more.”

(Fiódor Dostoiévski)

RESUMO

A simulação de corpos deformáveis — como tecidos, pele e materiais flexíveis — é um dos principais desafios da computação gráfica e é fundamental para aplicações interativas, como jogos digitais, realidade virtual, animação e simulações médicas. Diferentemente de corpos rígidos, esses objetos exigem modelagens que considerem deformações complexas e interações físicas realistas. No entanto, técnicas puramente físicas tendem a ser computacionalmente custosas, enquanto abordagens geométricas perdem em fidelidade. Este projeto de pesquisa propõe estudar e comparar métodos geométricos, físicos e híbridos de simulação de corpos e tecidos deformáveis, com foco em sua integração em motores de jogos como *Unity*, *Unreal* e *Godot*. A proposta inclui revisão de literatura, experimentação prática e análise comparativa de desempenho e realismo, com o objetivo de contribuir para o desenvolvimento de aplicações gráficas interativas mais precisas, imersivas e com melhor qualidade de experiência de usuário.

Palavras-chave: Corpos e Tecidos Deformáveis, Simulação Física, Motores de Jogos, Aplicações Gráficas Interativas.

ABSTRACT

The simulation of deformable bodies—such as cloth, skin, and flexible materials—represents one of the major challenges in computer graphics and is essential for applications in games, virtual reality, animation, and medical simulations. Unlike rigid bodies, these objects require models capable of handling complex deformations and realistic physical interactions. However, purely physics-based methods are often computationally expensive, while geometric approaches may lack physical fidelity. This work aims to study and compare geometric, physical, and hybrid methods for simulating deformable bodies and cloth, with a focus on their integration into game engines such as *Unity*, *Unreal*, and *Godot*. The proposal involves a literature review, practical experiments, and comparative analysis of performance and realism, with the aim of supporting the development of more immersive, accurate, and user-friendly interactive graphical applications.

Keywords: Deformable Bodies and Cloth, Physical Simulation, Game Engines, Interactive Graphics Applications.

LISTA DE ILUSTRAÇÕES

Figura 1 – Visão geral do trabalho de modelagem de tecidos.	15
Figura 2 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera totalmente deformada; e na terceira, já restituída, prestes a pular. . . .	39
Figura 3 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera parcialmente deformada; e na terceira, já completamente deformada e acoplada a outra esfera.	39
Figura 4 – Na primeira imagem, o instante antes da colisão; na segunda, a toalha parcialmente deformada; e na terceira, já completamente deformada e acoplada na mesa.	40
Figura 5 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera parcialmente deformada; e na terceira, já completamente deformada e acoplada a outra esfera.	41
Figura 6 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera totalmente deformada; e na terceira, já restituída, prestes a pular. . . .	42
Figura 7 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera parcialmente deformada; e na terceira, já completamente deformada e acoplada a outra esfera.	43
Figura 8 – Na primeira imagem, o instante antes da colisão; na segunda, a toalha parcialmente deformada; e na terceira, já completamente deformada e acoplada na mesa.	44
Figura 9 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera totalmente deformada; e na terceira, já restituída, prestes a pular. . . .	45
Figura 10 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera parcialmente deformada; e na terceira, já completamente deformada e acoplada a outra esfera.	45
Figura 11 – Na primeira imagem, o instante antes da colisão; na segunda, a toalha parcialmente deformada; e na terceira, já completamente deformada e acoplada na mesa.	46
Figura 12 – Fluxo do <i>Blueprint</i> para a simulação de corpos deformáveis.	55
Figura 13 – Propriedades do corpo deformável no OMNIVERSE	57
Figura 14 – Propriedades do Tecido no OMNIVERSE	58

SUMÁRIO

1	INTRODUÇÃO	11
1.1	OBJETIVOS	12
1.1.1	Objetivo Geral	12
1.1.2	Objetivos Específicos	12
1.2	ESTRUTURA DO TRABALHO	13
2	FUNDAMENTAÇÃO TEÓRICA	14
2.1	TÉCNICAS DE MODELAGEM DE TECIDOS	14
2.1.1	Técnicas Geométricas	14
2.1.2	Técnicas Baseadas em Física	16
2.1.3	Técnicas Híbridas	18
2.2	SIMULAÇÃO FÍSICA DE MODELOS ESTRUTURAIS	19
2.2.1	Sistemas Massa-Mola (<i>Mass-Spring Systems</i>)	19
2.2.2	Sistema Massa-Mola-Amortecedor (MSD - <i>Mass-Spring-Damper System</i>)	20
2.2.3	Método dos Elementos Finitos (FEM - <i>Finite Element Method</i>)	22
2.2.4	Métodos Baseados em Partículas (SPH - <i>Smoothed Particle Hydrodynamics</i>)	22
3	FUNDAMENTAÇÃO TECNOLÓGICA	24
3.1	GODOT	24
3.2	UNITY	25
3.3	UNREAL ENGINE	27
3.3.1	<i>Chaos Flesh</i>	28
3.3.2	<i>Chaos Cloth</i>	28
3.4	OMNIVERSE	29
3.5	DISCUSSÃO FINAL	31
4	TRABALHOS RELACIONADOS	32
4.1	ABORDAGENS GEOMÉTRICAS	32
4.2	MODELOS FÍSICOS BASEADOS EM MALHAS	32
4.3	MODELOS HÍBRIDOS E ELEMENTOS FINITOS	33
4.4	MODELAGEM COM SPH E CFD	33
4.5	CONSIDERAÇÕES FINAIS	34

5	METODOLOGIA	35
6	CASOS EXPERIMENTAIS	38
6.1	GODOT	39
6.1.1	Simulação de acoplamento entre esferas	39
6.1.2	Simulação da toalha sobre uma mesa	39
6.1.3	Medidas de desempenho	40
6.2	UNITY	41
6.2.1	Simulação de acoplamento entre esferas	41
6.2.2	Simulação da toalha sobre uma mesa	41
6.2.3	Medidas de desempenho	41
6.3	UNREAL ENGINE	42
6.3.1	Simulação de acoplamento entre esferas	42
6.3.2	Simulação da toalha sobre uma mesa	43
6.3.3	Medidas de desempenho	43
6.4	OMNIVERSE	44
6.4.1	Simulação de acoplamento entre esferas	44
6.4.2	Simulação da toalha sobre a mesa	45
6.4.3	Medidas de desempenho	45
6.5	DISCUSSÃO FINAL	46
7	CONCLUSÃO E TRABALHOS FUTUROS	48
	REFERÊNCIAS	50
	ANEXOS	52
	ANEXO A – CÓDIGO DE EXEMPLO DO SOFTBODY NA GODOT	53
	ANEXO B – CÓDIGO DE EXEMPLO DO SOFTBODY NA UNITY	54
	ANEXO C – CÓDIGO DE EXEMPLO DO SOFTBODY NA UNREAL ENGINE	55
	ANEXO D – CÓDIGO PARA GERAR A PLANE MESH NA OMNI- VERSE	56
	ANEXO E – CÓDIGO DE EXEMPLO DE CORPO DEFORMÁVEL OMNIVERSE	57
	ANEXO F – CÓDIGO DE EXEMPLO DE TECIDO NO OMNIVERSE	58

1 INTRODUÇÃO

Diferentemente de corpos rígidos, cujo movimento pode ser descrito predominantemente por translações e rotações, os *soft bodies* (corpos deformáveis) - incluindo a simulação de tecidos (*cloth* - exigem a modelagem de deformações complexas, como compressão, torção, estiramento e vibração, que envolvem interações dinâmicas entre partículas e estruturas internas. Essa categoria de simulação é fundamental em aplicações que vão desde efeitos especiais em filmes e jogos até modelos biomédicos e robótica flexível (AUBEL; THALMANN, 2000).

A representação realista de objetos deformáveis em ambientes virtuais constitui um dos maiores desafios da computação gráfica contemporânea. Tecidos, peles, órgãos e corpos moles devem, idealmente, comportar-se de maneira fisicamente plausível, respondendo corretamente a forças externas, colisões e limites geométricos, sem comprometer a performance necessária para aplicações interativas, tais como games, realidade virtual, videochamadas médicas e simulações científicas (BARAFF; WITKIN, 2023; FENG; YU; KIM, 2010).

Por outro lado, técnicas puramente físicas (baseadas em elementos finitos ou malhas massa-mola) frequentemente exigem recursos computacionais elevados e calibração complexa de parâmetros mecânicos (BARAFF; WITKIN, 2023; GOLDENTHAL *et al.*, 2007), enquanto abordagens geométricas ou orientadas por dados, embora eficientes, deixam de capturar nuances de elasticidade, inextensibilidade e auto-interseção (GOLDENTHAL *et al.*, 2007; HARADA; KOSHIZUKA; KAWAGUCHI, 2007). Dessa forma, surgem diversas estratégias híbridas que buscam equilibrar realismo físico e desempenho em tempo real, utilizando, por exemplo, modelos de baixa resolução para cálculo bruto de deformações e transformações orientadas por aprendizado para recuperar detalhes finos em malhas de alta resolução (FENG; YU; KIM, 2010).

Neste contexto, estudos recentes apresentam soluções que variam de projeções rápidas de restrições inextensíveis (eliminando “super-elasticidade” sem aumentar excessivamente a rigidez) (GOLDENTHAL *et al.*, 2007; BARAFF; WITKIN, 2023) a *pipelines* que exploram arquiteturas paralelas, implantando intermediários de simulação em GPUs, utilizando estruturas de grade para busca de vizinhança em fluidos ou partículas (HARADA; KOSHIZUKA; KAWAGUCHI, 2007), ou mesmo combinando *ray marching* com campos implícitos (*Metaball*) para representar corpos moles em motores de jogos (KIM, 2022).

Há, igualmente, pesquisas aplicadas ao domínio médico, em que a Dinâmica Baseada em Posições (PBD) é empregada para simular deformação de órgãos em realidade aumentada, fornecendo *feedback* háptico e visual para procedimentos cirúrgicos (FENG; YU; KIM, 2010).

Apesar dessa variedade de métodos, permanece a necessidade de avaliar e comparar rigorosamente cada estratégia em termos de fidelidade mecânica, custo computacional, robustez frente a colisões e escalabilidade em arquiteturas *multicore* e GPU (NIKOOYAN; ZADPOOR, 2011).

Por fim, a modelagem e simulação de corpos e tecidos deformáveis desempenham papel central em aplicações gráficas interativas, como jogos digitais, realidade virtual, simulações médicas, treinamentos e animações computacionais. Esses elementos aumentam a credibilidade visual e a resposta física de objetos moles, roupas, tecidos ou superfícies elásticas em contato com personagens e ambientes. Com o avanço das *engines* gráficas como *Unity* (UNITY-TECHNOLOGIES, 2025), *Unreal* (EPIC-GAMES, 2025) e *Godot* (GODOT, 2025), tornou-se viável integrar muitas destas simulações em tempo real, exigindo técnicas otimizadas e realistas.

Este projeto de pesquisa propõe estudar e comparar métodos de modelagem e simulação de corpos deformáveis, explorando abordagens geométricas e físicas, com foco em sua aplicação em ambientes interativos. O objetivo é apoiar o desenvolvimento de experiências mais imersivas e precisas, que combinam desempenho computacional com fidelidade visual.

1.1 OBJETIVOS

1.1.1 Objetivo Geral

O objetivo geral deste projeto é investigar as principais técnicas de modelagem e simulação física de tecidos e corpos deformáveis em motores de jogos e motores de física para aplicá-las no desenvolvimento de aplicações gráficas interativas.

1.1.2 Objetivos Específicos

Os objetivos específicos deste projeto são os seguintes:

- Avaliar detalhadamente as principais técnicas na área já propostas no passado e as mais recentemente utilizadas.
- Identificar como as principais *game engines* atuais lidam com corpos e tecidos deformáveis.
- Realizar testes comparativos utilizando casos de estudos implementados em diferentes *game engines*, no caso, *Godot*, *Unity*, e *Unreal*, a fim de identificar suas potencialidades e limitações práticas.

1.2 ESTRUTURA DO TRABALHO

O Capítulo 2 descreve as técnicas de simulação de corpos deformáveis, incluindo os modelos massa-mola, massa-mola-amortecedor, o método de elementos finitos (FEM) e métodos baseados em partículas (GOLDENTHAL *et al.*, 2007). O Capítulo 3 aborda como as principais plataformas do mercado implementam *soft bodies*. A análise inclui três *game engines* proeminentes — *Godot 4* (GODOT, 2025), *Unity 6* (UNITY-TECHNOLOGIES, 2025) e *Unreal Engine 5* (EPIC-GAMES, 2025) — e a plataforma de simulação e colaboração *NVIDIA Omniverse* (NVIDIA, 2025b). O Capítulo 4 analisa e compara publicações relevantes da área, destacando seus principais méritos e limitações. O Capítulo 5 detalha os passos definidos para o desenvolvimento deste trabalho. O Capítulo 6 detalha os casos experimentais. São apresentados testes comparativos realizados nas plataformas investigadas (*game engines* e *NVIDIA Omniverse*) para conduzir experimentos com modelos de corpos moles. Por fim, o Capítulo 7 sintetiza os aprendizados obtidos e propõe direções para estudos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Serão abordadas duas frentes principais neste capítulo: técnicas de modelagem de tecidos, com foco em algoritmos específicos para representar malhas do tipo *cloth* e suas propriedades elásticas e de cisalhamento (NG; GRIMSDALE, 1996); e simulação física de materiais (NIKOOYAN; ZADPOOR, 2011), com a apresentação de modelos matemáticos e numéricos que descrevem a resposta de diferentes materiais deformáveis sob a ação de forças externas.

Ao longo das próximas seções, serão apresentados a base matemática necessária e os fundamentos das principais pesquisas analisadas no desenvolvimento deste trabalho de conclusão de curso.

2.1 TÉCNICAS DE MODELAGEM DE TECIDOS

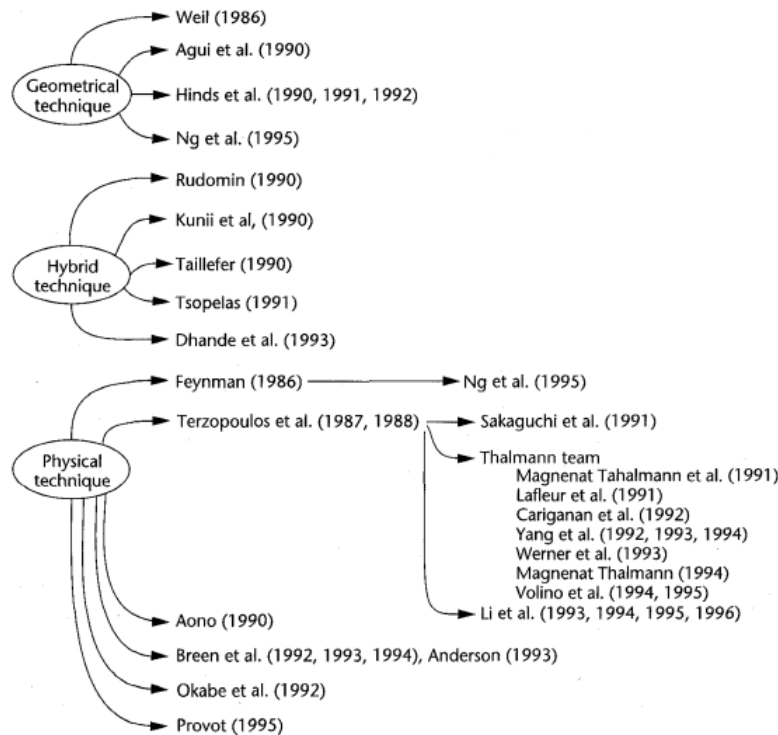
Esta seção apresenta uma visão geral das principais técnicas desenvolvidas ao longo dos anos para modelagem de tecidos, com base em uma revisão abrangente da literatura sobre o tema (NG; GRIMSDALE, 1996). A Figura 1 ilustra uma visão geral das principais contribuições na área.

2.1.1 Técnicas Geométricas

Os modelos geométricos de tecido simplificam a simulação ao ignorar as leis físicas que governam o comportamento real do plano. Em vez disso, concentram-se apenas em como o tecido parece - principalmente em como se formam dobras e vincos - e descrevem esses detalhes por meio de equações matemáticas. Por tratarem mais de desenho avançado do que de física, esses métodos costumam exigir bastante intervenção manual do usuário para ajustar formas e posicionamentos, funcionando basicamente como uma ferramenta de desenho especializada.

Weil *et al.* (NG; GRIMSDALE, 1996) foi um dos primeiros a representar tecidos usando apenas geometria, modelando este tipo de objeto como uma malha de pontos conectados por curvas de catenária, que imitam o caimento natural de cabos pesados. Curvas inferiores eram removidas em interseções no plano x - z , criando um esqueleto curvado. A malha era então refinada com subdivisões e ajustes para simular dobras, suavizada com splines e tratada como objetos cilíndricos para ray tracing. Apesar de ser rápido e eficiente, o modelo se limita a tecidos pendurados e desconsidera propriedades físicas como massa, elasticidade e atrito.

Figura 1 – Visão geral do trabalho de modelagem de tecidos.



Fonte: Extraído de (NG; GRIMSDALE, 1996)

Agui *et al.* (NG; GRIMSDALE, 1996) propõem um método geométrico para modelar a manga de uma camisa em um braço dobrado, representando o tecido como um cilindro oco formado por anéis circulares. As dobras são associadas à diferença de curvatura entre as faces interna e externa da manga, sendo identificadas por variações mínimas na distância $dL(i, j)$ entre o tecido e o braço após a deformação. Os anéis são convertidos em polígonos com dobras, assumindo simetria em torno do eixo y e restringindo os vincos à parte superior da manga. Para suavizar os ângulos formados, cada vértice é reposicionado com base na média dos seus vizinhos. Embora eficaz nesse contexto, o método não especifica os critérios exatos para detecção das dobras.

Hinds *et al.* (NG; GRIMSDALE, 1996) propõem um sistema para automação do *design* de roupas baseado em modelos 3D do corpo humano, obtidos via digitalização. As peças são compostas por painéis tridimensionais ajustados manualmente sobre o corpo digital, com bordas definidas como deslocamentos da superfície. Os demais pontos são gerados por interpolação, garantindo continuidade entre painéis. Dobras e texturas podem ser adicionadas com funções harmônicas. Em trabalhos posteriores, os autores mostram como transformar esses painéis em moldes 2D e aprimoram as ferramentas interativas de *design*.

Ng e Grimsdale (1996) descrevem um método de modelagem para animação que vincula diretamente a camada de tecido à forma da pele subjacente, ambas organizadas em seções paralelas com o mesmo número de vértices (NG; GRIMSDALE, 1996). As regiões folgadas são identificadas automaticamente pelas lacunas entre as camadas, e linhas de dobra são geradas e modificadas por funções senoidais para criar ondulações realistas. O editor interativo Geoff permite ao usuário ajustar essas linhas com base em regras estruturais, atribuindo propriedades visuais e animáveis. O método, por evitar equações complexas, é adequado para aplicações em tempo real.

2.1.2 Técnicas Baseadas em Física

As técnicas baseadas em Física modelam o tecido como uma malha (triangular ou retangular) cujos vértices são pontos com massa finita ligados uns aos outros, e calculam as forças ou energias desses pontos em função de seus vizinhos. Dependendo do método, cada ponto interage com um número diferente de vizinhos, mas o princípio geral é o mesmo: técnicas baseadas em energia definem uma função de energia para todo o tecido e movem os pontos até que essa energia seja minimizada, produzindo simulações estáticas; já as técnicas baseadas em força formulam equações diferenciais que descrevem as forças atuantes entre os pontos e, por meio de integração numérica em cada passo de tempo, de forma realista, o comportamento flexível do pano, embora o grau de fidelidade e o custo computacional variem conforme o modelo adotado.

Feynman (NG; GRIMSDALE, 1996) propôs um modelo físico para tecidos drapeados, representando o pano como uma malha bidimensional no espaço 3D, com cada ponto interagindo com seus oito vizinhos. Com base na teoria de placas elásticas, definiu a energia existente em cada vértice como a soma ponderada de termos de elasticidade, curvatura e gravidade. A forma final do tecido é obtida minimizando essa energia via gradiente descendente, acelerado por técnica de multigrid. O método é aplicado a simulações de tecido pendurado e cobrindo uma esfera, sem detalhar o tempo de execução.

Aono (NG; GRIMSDALE, 1996) propôs um modelo matemático para a propagação de rugas em tecidos, tratando-os como meios elásticos homogêneos e isotrópicos sob pequenas deformações. Com base na elasticidade linear e no princípio de D'Alembert, formulou uma equação diferencial parcial que combina termos de inércia, rigidez e amortecimento viscoso. Para facilitar a análise, assumiu ausência de deslocamento entre fibras e incorporou efeitos visco-

elásticos via ajuste de parâmetros. O modelo foca exclusivamente nas ondulações propagadas, desconsiderando atrito, arrasto e contato com o corpo.

O grupo da Universidade de Quioto (NG; GRIMSDALE, 1996) desenvolveu o sistema *Party17* para simular tecidos com base em uma malha de pontos de massa interconectados, regidos pela segunda lei de Newton. As forças internas combinam elasticidade, viscosidade e plasticidade, permitindo coesão, amortecimento e deformações permanentes. A simulação temporal é feita via método de Euler, e colisões com corpos virtuais são tratadas com conservação de momento e volumes limitadores para reduzir o custo computacional. O modelo inclui atrito, promovendo interações realistas com superfícies.

Thalmanns *et al.* (NG; GRIMSDALE, 1996) propuseram uma abordagem de simulação de roupas baseada no processo real de confecção: modelagem de painéis planos, costura tridimensional e animação da interação com o corpo virtual.

Para colisões, Lafleur (NG; GRIMSDALE, 1996) usou campos de força repulsivos, enquanto Yang otimizou a detecção com caixas delimitadoras e árvores octais, ainda com alto custo computacional.

Carignan (NG; GRIMSDALE, 1996) incorporou um modelo dinâmico com tensões elásticas, viscosas e plásticas, além de colisões inelásticas e restrições de costura.

Volino (NG; GRIMSDALE, 1996) propôs um método em duas etapas, tratando forças e colisões separadamente, com hierarquização da malha para melhorar o desempenho.

Breen *et al.* (NG; GRIMSDALE, 1996) desenvolveram um método para simular o caimento de tecidos entrelaçados, modelando cada cruzamento entre urdume e trama como uma partícula sob forças físicas. Inicialmente, essas partículas sofrem ação da gravidade e resistência do ar, gerando uma forma preliminar ao repousar sobre superfícies. Em seguida, aplicam minimização de energia com termos que penalizam repulsão, alongamento, dobras e gravidade, resolvida via Métodos Estocásticos, que exigiam dias de processamento na época.

Anderson (NG; GRIMSDALE, 1996) posteriormente acelerou a simulação ao implementá-la em supercomputadores paralelos, reduzindo o tempo para poucas horas e revelando que o principal limitante era computacional.

Okabe *et al.* (NG; GRIMSDALE, 1996) desenvolveram um sistema que automatiza a confecção de roupas a partir de moldes 2D, escaneando-os e projetando-os sobre um manequim virtual. O padrão é inicialmente esticado sobre uma malha triangular no corpo, sem considerar o comportamento físico do tecido. Para simular o caimento, aplicam teoria da elasticidade de cascas, atribuindo energia a cada nó da malha, considerando alongação, cisalhamento, dobra,

torção, gravidade e penalidade por penetração. Um algoritmo adaptativo de descida de gradiente ajusta as posições até atingir o equilíbrio, em cerca de 20 minutos com 1.300 nós. O sistema também reconstrói um molde 2D refinado com base na superfície ajustada.

Li *et al.* (NG; GRIMSDALE, 1996) modelaram a interação entre tecidos e vento tratando o pano como uma superfície impermeável segmentada em *patches*, sobre os quais aplicaram as equações de Navier-Stokes médias de Reynolds para calcular forças aerodinâmicas. Essas forças alimentam um modelo deformável baseado em Terzopoulos (NG; GRIMSDALE, 1996), que atualiza os vértices do tecido com base em tensões elásticas e viscosas. Versões posteriores incluíram fluxos não estacionários para lidar com variações rápidas no ar, com maior custo computacional, gerando simulações realistas como bandeiras ondulando ao vento.

Provot (NG; GRIMSDALE, 1996) propôs um modelo clássico de molas e massas para simulação de tecidos, tratando cada ponto como uma massa submetida às leis de Newton e conectada por molas estruturais, de flexão e de cisalhamento. Essas molas geram forças internas proporcionais ao seu alongamento, somadas à gravidade, amortecimento e viscosidade do ar. Utilizando o método de Euler para integração temporal, o autor enfrentou o problema de alongamentos excessivos em pontos de fixação e propôs um limite de deformação de 10% para as molas nesses pontos, evitando instabilidades sem aumentar demasiadamente a rigidez. O modelo mostrou-se eficaz em testes como bandeiras ao vento, com boa relação entre realismo e custo computacional.

Ng *et al.* (NG; GRIMSDALE, 1996) desenvolveram um sistema de visualização de tecidos baseado na minimização de energia, com funções adaptadas do modelo de Feynman (NG; GRIMSDALE, 1996). As geometrias geradas são exportadas em formato DXF, compatível com ferramentas CAD.

2.1.3 Técnicas Híbridas

Técnicas híbridas combinam métodos físicos e geométricos.

Rudomin (NG; GRIMSDALE, 1996) propôs acelerar a simulação de drapeados de tecido por meio de uma aproximação geométrica inicial. O método envolve o objeto com um pano virtual, calcula seu invólucro convexo e remove a parte inferior com um plano abaixo do centro de massa. A parte superior é então projetada no domínio bidimensional do pano, descartando pontos fora dos limites reais. Os pontos restantes são reconstruídos em 3D para formar um envoltório inicial do tecido, que serve como estimativa para um *solver* tradicional,

como o modelo de Terzopoulos (NG; GRIMSDALE, 1996), para refinar o drapeado.

Kunii *et al.* (NG; GRIMSDALE, 1996) propuseram um método híbrido para simulação de tecidos que combina física e geometria, buscando contornar a lentidão e dificuldade de parametrização dos modelos puramente físicos implementados na época. Na primeira etapa, realizaram uma simulação física tradicional utilizando uma malha estrutural composta de molas e pontos, minimizando energias métricas e de curvatura. Depois, extraíram pontos característicos e contornos de dobras reconstruídas geometricamente por curvas senoidais parametrizadas pelo comprimento de arco e aplicadas ao vestuário, com minimização de energia para ajustar o modelo final. Apesar dos bons resultados visuais, o método assume número fixo de rugas e ignora a interação com a superfície subjacente.

Taillefer (NG; GRIMSDALE, 1996) estendeu o modelo de Weil (NG; GRIMSDALE, 1996) para tratar especificamente de tecidos suspensos, distinguindo duas categorias de pregas: horizontais e verticais. As pregas horizontais são descritas por curvas catenárias ¹.

Dhande *et al.* (NG; GRIMSDALE, 1996) propuseram um método híbrido baseado em “varredura” de superfícies para simular o caimento de um tecido sobre um objeto rígido. A ideia central é gerar uma superfície biparamétrica a partir de uma curva diretora $r_2(v)$, que segue a geometria do objeto, e de uma curva geratriz $r_1(u, v)$, que descreve a forma do tecido fora do objeto, aplicando-se uma transformação $M(v)$ que leva as normais locais da geratriz para o sistema global de coordenadas. Para modelar o comportamento físico do tecido, a geratriz é obtida a partir de uma “elástica” que incorpora rigidez à flexão e ao cisalhamento do pano. Embora a técnica gere resultados convincentes para tecidos drapeados sobre superfícies planas, os autores reconhecem que ela se complica muito quando o suporte tem geometria complexa, devido às dificuldades de manter a superfície varrida sem auto-interseções, nem distorções.

2.2 SIMULAÇÃO FÍSICA DE MODELOS ESTRUTURAIS

2.2.1 Sistemas Massa-Mola (*Mass-Spring Systems*)

O modelo apresentado em (PROVOT *et al.*, 1995) organiza $m \times n$ massas pontuais dispostas em uma malha regular, em que cada ponto P_{ij} está ligado a seus vizinhos por molas sem massa, de diferentes tipos. As molas “estruturais” conectam cada ponto ao vizinho imediato

¹ As curvas catenárias são formas matemáticas que descrevem a maneira como um fio ou corrente pesada assume uma forma curva quando pendurado por dois pontos, sob a ação da gravidade. No contexto da computação gráfica, elas são usadas para modelar a aparência de objetos flexíveis, como tecidos pendurados. (NG; GRIMSDALE, 1996)

na horizontal e na vertical, garantindo resistência à tração e à compressão em direções ortogonais. As molas “de cisalhamento” unem cada ponto às diagonais da malha, conferindo rigidez contra deformações oblíquas, e as molas “de flexão” ligam pontos separados por dois passos na mesma linha ou coluna, introduzindo resistência ao dobramento da superfície simulada.

A dinâmica de cada massa resulta do somatório das forças internas - originadas das tensões elásticas em todas as molas que a ligam aos seus vizinhos, proporcionais à diferença entre o comprimento atual da mola e seu comprimento natural, vezes sua constante de rigidez - e das forças externas, como a gravidade, um amortecimento viscoso que dissipa energia proporcional à velocidade do ponto, e eventualmente um arrasto de fluido, que modela o atrito com o ar ou outro meio. Essas forças aplicam ao ponto P_{ij} a resultante $F_{ij} = ma_{ij}$, de onde se obtém sua aceleração a_{ij} .

Para avançar no tempo, o modelo usa o método explícito de *Euler* (PROVOT *et al.*, 1995): a cada passo de duração Δt , calcula-se primeiro a nova aceleração a partir da força total, depois atualizam-se a velocidade, e finalmente, a posição de cada massa. Embora simples de implementar, esse esquema requer passos de tempo pequenos para manter a simulação estável.

Em cenários nos quais certos pontos são mantidos em posições ou velocidades predefinidas (como cantos de um tecido presos a hastes rígidas) ou quando ocorrem colisões com outros objetos, aplica-se uma técnica de dinâmica inversa. Nela, força-se determinados vértices da malha a seguir trajetórias impostas e, a partir daí, determinam-se indiretamente as forças de reação necessárias nas molas para manter a consistência física do conjunto.

Com essa combinação de tipos de mola, modelagem de forças internas e externas, e integração numérica explícita, o sistema de massa-mola consegue reproduzir de forma eficiente e controlável comportamentos que vão desde o escoamento suave de um tecido até a resistência ao dobramento e à tração de superfícies mais rígidas.

2.2.2 Sistema Massa-Mola-Amortecedor (MSD - *Mass-Spring-Damper System*)

O modelo de massa-mola-amortecedor (*Mass-Spring-Damper, MSD*) como apresentado no artigo (NIKOOYAN; ZADPOOR, 2011) é uma abstração matemática amplamente usada em engenharia, física computacional e até em animação por computador para representar sistemas dinâmicos de forma simples, porém, capaz de capturar os comportamentos de muitos materiais e estruturas. A ideia básica é decompor um objeto ou sistema em três componentes fundamentais:

- Massa(*Mass*): cada bloco ou elemento de massa m armazena energia cinética e responde à aceleração de acordo com segunda lei de Newton, $m\ddot{y} + k_{very} = mg$.
- Mola (*Spring*): cada conexão elástica, de constante de rigidez k , armazena energia potencial elástica e gera uma força proporcional ao deslocamento relativo Δx entre seus extremos, $F_{spring} = -k\Delta x$.
- Amortecedor (*Damper*): cada elemento viscoso, de coeficiente de amortecimento c , dissipa energia e produz uma força proporcional à velocidade relativa $\Delta \dot{x}$, $F_{damper} = -c\Delta \dot{x}$.

Quando massas, molas e amortecedores são conectados em uma rede (com um único grau de liberdade, até centenas ou milhares, no caso de malhas para simulação de tecidos ou estruturas deformáveis complexas), gera-se um sistema de equações diferenciais acopladas, como mostra a Equação 2.1:

$$M\ddot{x} + C\dot{x} + Kx = F(t), \quad (2.1)$$

- $\mathbf{x}$ é o vetor de deslocamento de cada massa,
- $\mathbf{M}$ é a matrix diagonal (ou em blocos) de massas,
- $\mathbf{C}$ contém os coeficientes de amortecimento,
- $\mathbf{K}$ reúne as constantes das molas,
- $\mathbf{F}(t)$ são forças externas aplicadas (por exemplo, gravidade, impactos ou cargas temporárias).

Na prática, esse modelo permite:

- Analisar vibrações de pontes, edifícios ou componentes mecânicos, para garantir que frequências de ressonância fiquem fora de faixas perigosas;
- Simular o comportamento de materiais não-lineares, bastando ajustar k e c em funções de deslocamento e velocidade;
- Projetar sistemas de suspensão em veículos, equilibrando conforto (amortecimento) e robustez (rigidez);
- Animar objetos em jogos e filmes, atribuindo propriedades físicas que tornam movimentos naturais sem precisar resolver equações completas da elasticidade contínua.

Embora o exemplo clássico seja o corpo humano correndo – onde massa representa segmentos corporais, mola reflete a rigidez combinada de ossos, tendões e músculos, e amortecedor modela a dissipação de energia em tecidos moles, esta técnica vale para qualquer sistema

que carregue inércia, elasticidade e atrito viscoso. A chave é escolher o número de massas e a topologia de molas/amortecedores de modo a capturar o fenômeno de interesse, manter o modelo simples o bastante para ser resolvido em tempo razoável e ajustar os parâmetros (m , k , c), a partir de experimentos ou dados de materiais.

2.2.3 Método dos Elementos Finitos (FEM - *Finite Element Method*)

Cada vértice da malha armazena sua posição local e valores de deslocamento, assumindo-se que o material entre eles se comporta, em média, conforme as leis físicas originais. O Método dos Elementos Finitos (FEM) transforma esse problema contínuo em um sistema discreto de equações algébricas, conectando cada nó aos seus vizinhos imediatos. Isso permite calcular forças resultantes e acelerações a cada passo de tempo de forma estável e computacionalmente eficiente.

Para derivar as forças internas a partir dos deslocamentos dos nós, utiliza-se uma formulação de elementos finitos baseada no princípio variacional, assumindo um campo de deslocamento linear em cada tetraedro. Esse processo converte integrais volumétricas em integrais de contorno via o Teorema de Gauss (DEBUNNE *et al.*, 2000). Assim, a força em um nó depende da diferença de deslocamento entre vizinhos, ponderada por essas cotangentes. A abordagem híbrida Volumes Finitos / Elementos Finitos proposta no artigo elimina termos redundantes e instabilidades na fronteira, mantendo a precisão do FEM com o desempenho computacional próximo ao de um modelo massa-mola. A implementação envolve hierarquia de malhas, mas o cálculo local segue a mesma lógica em todas as resoluções. No geral, o FEM transforma o problema contínuo em um sistema discreto de equações, permitindo simulação eficiente e estável de objetos deformáveis em tempo real.

2.2.4 Métodos Baseados em Partículas (SPH - *Smoothed Particle Hydrodynamics*)

Este método, descrito em (HARADA; KOSHIZUKA; KAWAGUCHI, 2007), simula o comportamento de fluidos utilizando partículas independentes em vez de malhas ou grades fixas. Cada partícula representa uma pequena porção do fluido e armazena informações como densidade e velocidade. O comportamento de uma partícula é influenciado apenas por suas vizinhas mais próximas, conforme determinado por um *kernel* de interpolação que atribui pesos maiores a partículas próximas e decresce suavemente até zero fora de um raio definido. Essa abordagem permite modelar fluidos de forma flexível e eficiente, especialmente em cenários

com grandes deformações ou mudanças topológicas.

Quando o fluido se movimenta ou é pressionado, cada partícula calcula localmente a densidade ao seu redor com base na contribuição das vizinhas próximas, estimando assim a pressão e a direção da força resultante. A viscosidade, por sua vez, é obtida comparando as velocidades entre partículas vizinhas, gerando forças que promovem o equilíbrio, como em líquidos reais. Para tornar o processo eficiente mesmo com milhares de partículas, o espaço é dividido em regiões (ou “baldes”), e cada partícula interage apenas com aquelas em sua própria célula e nas adjacentes, evitando buscas globais. Essa organização espacial é implementada na GPU via mapas de textura, permitindo acesso paralelo e acelerado. Barreiras físicas são representadas por partículas especiais de parede, que geram forças suaves de repulsão quando uma partícula se aproxima da borda. Todo o ciclo – densidade, pressão, viscosidade e movimento – é repetido a cada passo de tempo, geralmente via método explícito como o de Euler (HARADA; KOSHIZUKA; KAWAGUCHI, 2007), garantindo uma simulação estável desde que respeitado o critério de estabilidade numérica.

3 FUNDAMENTAÇÃO TECNOLÓGICA

Neste capítulo será introduzida a fundamentação tecnológica relevante para esta pesquisa, com foco nas funcionalidades disponíveis nos principais ambientes de simulação em tempo real da atualidade. Mais especificamente, este estudo envolve o conhecimento sobre simulações com *cloths* (tecidos deformáveis) e *soft bodies* (corpos deformáveis) (NG; GRIMSDALE, 1996), com foco nas soluções ora disponíveis em três importantes *engines* de jogos — *Godot* (GODOT, 2025), *Unity* (UNITY-TECHNOLOGIES, 2025) e *Unreal Engine* (EPIC-GAMES, 2025) — e na plataforma de simulação industrial *NVIDIA Omniverse* (NVIDIA, 2025b), notável por seu foco em fidelidade física.

3.1 GODOT

Na *Godot 4*, a simulação de *soft bodies* é viabilizada por meio do nó *SoftBody3D*, desenvolvido para representar objetos deformáveis com um comportamento físico realista e dinâmico (GODOT, 2025). Esse recurso permite a criação de corpos que reagem a forças externas e colisões de forma maleável, sendo útil em cenários que exigem física avançada, como tecidos, gelatinas ou superfícies elásticas.

Com o lançamento da versão 4, a *Godot* passou a incorporar de forma nativa parte do motor físico *Jolt Physics*, que anteriormente era disponibilizado apenas como um *plugin* de terceiros. Além disso, a engine mantém integração com o *Bullet Physics* (COUMANS, 2025), oferecendo uma base sólida para a simulação de interações físicas complexas e flexíveis dentro do ambiente 3D.

Apesar desses avanços, é fundamental destacar uma limitação importante registrada na própria documentação oficial da *Godot*: o nó *SoftBody3D* ainda apresenta diversos *bugs* conhecidos e instabilidades. Por esse motivo, os desenvolvedores da *engine* recomendam evitar sua utilização em elementos críticos de jogabilidade, como trampolins, plataformas móveis ou quaisquer componentes que possam comprometer a progressão do jogador. Essas falhas podem causar comportamentos inesperados, prejudicando a experiência do usuário e, em casos mais severos, comprometendo o funcionamento de mecânicas essenciais dentro do jogo (GODOT, 2025).

Propriedades físicas ajustáveis: O nó permite configurar diversos parâmetros físicos que influenciam o comportamento da simulação, como:

- **Simulation Precision:** Define o número de interações da simulação física por quadro.
- **Total Mass:** Determina a massa total do objeto, influenciando sua inércia, resposta à gravidade e resistência a forças externas.
- **Linear Stiffness:** Controla a rigidez do material, regulando a resistência do objeto à deformação.
- **Pressure Coefficient:** Define a pressão interna do objeto, afetando seu grau de inflação e a rigidez estrutural.
- **Damping Coefficient:** Estabelece a resistência ao movimento, simulando o arrasto que reduz a velocidade do objeto ao se mover.
- **Ray Pickable:** Indica se o objeto pode ser selecionado por meio de interações com raios, como cliques ou seletores.
- **Disable Mode:** Especifica que o objeto será removido da simulação física ao ser desativado, otimizando o uso de recursos.
- **Pinned Points:** Define pontos fixos da malha que permanecem estáticos durante a simulação, funcionando como ancoragens no espaço.

Integração com o sistema de física: O *SoftBody3D* interage com os demais elementos físicos da cena, permitindo colisões e interações realistas com outros corpos. Você pode, inclusive, combinar a malha deformável com colisores adicionais para obter simulações mais precisas.

Personalização via scripts: Além dos ajustes diretos na interface do nó, é possível controlar e modificar os parâmetros do *soft body* por meio de *scripts* (GDScript ou C#), permitindo efeitos personalizados ou reações específicas durante a execução do jogo.

3.2 UNITY

Na *Unity 6* (UNITY-TECHNOLOGIES, 2025), embora não exista um componente nativo dedicado exclusivamente a *soft bodies*, é possível implementar esse tipo de simulação utilizando ferramentas integradas, *assets* de terceiros ou soluções personalizadas, combinando física e *scripts* (UNITY-TECHNOLOGIES, 2025).

Abordagens Principais:

- **Componente Cloth:** A Unity oferece o componente *Cloth*, projetado para simular tecidos e superfícies flexíveis. É fundamental notar que este componente requer obrigatoriamente um *SkinnedMeshRenderer* para funcionar, aplicando a simulação diretamente sobre a

malha “skinada”. Embora seja útil para roupas, bandeiras ou cortinas, ele possui limitações em simulações mais complexas (como objetos 3D volumétricos).

- **Obi SoftBody:** Pacotes como *Obi SoftBody* são soluções populares na *Asset Store*, oferecendo simulações realistas de *soft bodies* com suporte a colisões, pressão interna e deformações baseadas em vértices. É ideal para projetos que exigem resultados rápidos e profissionais.
- **Sistemas customizados com Molas e Partículas:** Desenvolvedores podem criar *soft bodies* manualmente usando partículas interconectadas por “molas” (*spring joints*) e forças aplicadas. Essa abordagem exige cálculos matemáticos para elasticidade, amortecimento e deformação, sendo mais flexível, porém complexa.
- **Position-Based Dynamics (PBD):** Algoritmos como PBD permitem simulações estáveis de deformações, ajustando posições diretamente para evitar instabilidades numéricas. Bibliotecas como *Verlet Integration* podem ser implementadas via *scripts*.

Propriedades físicas ajustáveis:

- **Stretching Stiffness:** Rigidez de alongamento do tecido.
- **Bending Stiffness:** Rigidez de flexão do tecido.
- **Use Tethers:** Aplica restrições que ajudam a evitar que o tecido se afaste demais da área fixa. Isso ajuda a reduzir restrições em excesso.
- **Use Gravity:** A gravidade influencia diretamente o caimento de um tecido, a formação de dobras e a interação do tecido com outros objetos. Contudo, pode-se optar por desativar esta opção, se necessário (por exemplo, em ambientes sem gravidade, suspensos artificialmente, ou controlado apenas por forças personalizadas, etc.).
- **Damping:** Coeficiente de amortecimento do movimento.
- **External Acceleration:** Uma aceleração externa constante aplicada a cada vértice do tecido.
- **Random Acceleration:** Uma aceleração externa aleatória aplicada a cada vértice do tecido.
- **World Velocity Scale:** Quanto o movimento em espaço global do personagem afetará os vértices do tecido.
- **World Acceleration Scale:** Quanto a aceleração em espaço global do personagem afetará os vértices do tecido.
- **Friction:** O atrito do tecido ao colidir com o personagem.
- **Collision Mass Scale:** Quanto aumentar a massa das partículas em colisão.

- ***Use Continuous Collision***: Ativa a colisão contínua para melhorar a estabilidade das colisões.
- ***Use Virtual Particles***: Adiciona uma partícula virtual por triângulo para melhorar a estabilidade das colisões.
- ***Solver Frequency***: Número de iterações do solucionador por segundo.
- ***Sleep Threshold***: Limite de velocidade abaixo do qual esta instância de tecido deve entrar em repouso.
- ***Capsule Colliders***: Uma lista (*array*) de *CapsuleColliders* com os quais esta instância de tecido deve colidir.
- ***Sphere Colliders***: Uma lista (*array*) de *ClothSphereColliderPairs* com os quais esta instância de tecido deve colidir.

Integração com o sistema de física: Com o uso da propriedade *Cloth* é possível fazer o objeto interagir com outras partes do cenário, mas vale lembrar que o *Cloth* só possui colisão com colidores esféricos e de cápsula.

Personalização via *scripts*: Além dos ajustes diretos no componente *Cloth* do objeto 3D, é possível controlar e modificar os parâmetros por meio de *scripts* em C#.

3.3 UNREAL ENGINE

Na Unreal Engine 5, o *Chaos Flesh* oferece simulação em tempo real de *soft bodies* ao converter *Static* e *Skeletal Meshes* em malhas tetraédricas volumétricas, aplicando *constraints* para controlar massa e amortecimento por partícula, tudo gerenciado por um *Solver Actor* que suporta *sub-stepping*, *threading* e *caching* para *playback* suave via *Sequencer* ou *Movie Render Queue Epic Games Developer*.

Já o *Chaos Cloth* é um *solver* de partículas integrado ao editor que permite autorar, pintar pesos e ajustar parâmetros como *EdgeStiffness*, *AreaStiffness* e *BendingStiffness* diretamente no *viewport*, com suporte a *LOD* (*Level of Detail* ou Nível de Detalhamento de um Objeto), integração com *Blueprints* (Linguagem Visual de programação da Unreal) para colisões e animações dinâmicas, e compatibilidade com *Nanite* para manter alta performance em jogos, VFX e simulações interativas (EPIC-GAMES, 2025).

3.3.1 *Chaos Flesh*

Propriedades físicas ajustáveis: O nó permite configurar diversos parâmetros físicos que influenciam o comportamento da simulação, como:

- ***Density***: Define a massa volumétrica do material (massa por unidade de volume).
- ***Vertex Stiffness***: Controla o quão rígida é a ligação de cada vértice à sua posição de repouso.
- ***Vertex Damping***: Determina o amortecimento do movimento dos vértices (semelhante a um atrito interno).
- ***Vertex Incompressibility***: Regula a preservação de volume local.
- ***Vertex Inflation***: Aplica uma pressão interna uniforme, expandindo levemente a malha de dentro para fora.
- ***Active***: Liga ou desliga a simulação naquela *asset*.

Integração com o Sistema de Física: O *Chaos Flesh* na *Unreal Engine* integra-se ao sistema de física *Chaos* para simular corpos moles e deformáveis, como músculos e pele. Ele converte malhas em uma representação geométrica tetraédrica que é usada para uma simulação física em tempo real. Isso permite que os personagens exibam movimentos musculares e reações de tecidos moles mais realistas a forças e colisões, com a simulação acompanhando as animações enquanto adiciona deformações secundárias dinâmicas baseadas em propriedades físicas.

Personalização via *scripts*: É possível criar personalizações através da ferramenta *Blueprint*.

3.3.2 *Chaos Cloth*

Propriedades físicas ajustáveis: O nó permite configurar diversos parâmetros físicos que influenciam o comportamento da simulação, como:

- ***Density***: Valor usado quando o *Mass Mode* está em *Density*, controlando a massa por unidade de área do tecido.
- ***Bending Stiffness***: Rigidez das restrições de dobra (*bending constraints*), definindo o quanto o tecido resiste a alterar o ângulo entre elementos conectados.
- ***Area Stiffness***: Rigidez das restrições de “*tether*” (*long-range attachments*), que puxam cada vértice em direção à sua posição original na malha *skinned*.
- ***Tether Scale***: Fator de escala que determina o limite máximo de afastamento permitido

pelos *tethers* antes de entrarem em ação.

- **Friction Coefficient:** Coeficiente de atrito em interações tecido-colisor, controlando quão “pegajoso” ou deslizante é o contato com superfícies rígidas.
- **Damping Coefficient:** Amortecimento global aplicado às velocidades das partículas de tecido, reduzindo oscilações e estabilizando a simulação.
- **Drag:** Coeficiente aerodinâmico de arrasto em cada partícula, simulando resistência do ar proporcional à velocidade do tecido.
- **Lift:** Coeficiente aerodinâmico de sustentação em cada partícula, permitindo que o tecido receba forças de “elevação” quando exposto ao vento.
- **Gravity Scale:** Fator de escala aplicado à gravidade mundial na simulação de tecido, modulando o peso aparente do material.

Integração com o sistema de Física: O *Chaos Cloth* na *Unreal Engine* é a solução para simular tecidos e vestuário, integrando-se profundamente com o sistema de física *Chaos*. Opera em um nível de partículas, onde as propriedades do tecido ditam seu movimento e interação com o ambiente e outros objetos físicos. Isso permite que as roupas colidam realisticamente com os corpos dos personagens e reajam a forças externas como vento e gravidade, com parâmetros ajustáveis para controle detalhado do comportamento do tecido.

Personalização via scripts: É possível criar personalizações através da ferramenta *Blueprint*.

3.4 OMNIVERSE

Em 2025, o ecossistema *Omniverse* da *NVIDIA* passou por uma mudança significativa com a descontinuação do seu *launcher*. O método recomendado agora consiste na compilação direta dos *templates* a partir do código-fonte, o que garante suporte multiplataforma (*NVIDIA*, 2025a). Para este projeto, foi utilizado o template *USD Composer*, que oferece integração nativa com *plugins* de física e suporte ao motor *PhysX 5*.

O *Omniverse* utiliza duas abordagens principais para *soft bodies*. Para corpos volumétricos, é empregado o algoritmo de elementos finitos (FEM) (DEBUNNE *et al.*, 2000). O método apresentado é o procedimento padrão atual para criar corpos deformáveis volumétricos, mas está em processo de depreciação. A *NVIDIA* está desenvolvendo um sistema mais robusto e eficiente para a criação de *soft bodies*.

Esta implementação exige a adição da extensão *Physics Bundle* e a configuração de

um *PhysicsScene* para habilitar interações fundamentais como gravidade e colisão.

Propriedades físicas (Deformable Body)

- **Density:** Define a massa do corpo por unidade de volume.
- **Dynamic Friction:** Controla o atrito em movimento entre o corpo e outros objetos.
- **Young's Modulus:** Determina a rigidez do material (resistência à deformação elástica).
- **Poisson's Ratio:** Descreve a tendência do material a se expandir ou contrair perpendicularmente à direção da compressão/tração.
- **Elasticity Damping:** Amortecimento da elasticidade, que dissipa a energia de vibração.
- **Damping Scale:** Fator de escala aplicado a todos os parâmetros de amortecimento.

Simulação de Tecidos (Particle Cloth)

Para a simulação de superfícies finas, como tecidos, o *OMNIVERSE* utiliza o componente *Particle Cloth*. Diferente do método FEM volumétrico, este sistema simula o material como uma grade de partículas interconectadas por restrições (similar a um sistema massa-mola avançado).

A configuração também requer um *PhysicsScene* e que os objetos de interação possuam *Colliders*. A simulação é ativada aplicando-se a propriedade *Particle Cloth* diretamente a uma *mesh* (tipicamente um *plane* com alta subdivisão).

Propriedades físicas ajustáveis (Particle Cloth)

- **Stretch Stiffness:** Define a rigidez do tecido, controlando o quanto ele resiste a ser esticado (resistência à tração).
- **Bend Stiffness:** Controla a resistência do material a dobras e vincos. Valores altos criam um material “duro” (como lona), enquanto valores baixos simulam tecidos mais fluidos (como seda).
- **Shear Stiffness:** Determina a resistência à distorção de cisalhamento (quando o tecido é puxado em direções opostas na diagonal).
- **Spring Damping:** Dissipa a energia do sistema de molas, controlando o quanto o tecido “oscila” antes de atingir o repouso após uma deformação.
- **Disable Mesh Welding:** Quando desabilitado (marcado), impede que o sistema una (solde) vértices da malha que estão muito próximos, tratando cada vértice de forma independente.

Integração com o sistema de Física: O *PhysX* é integrado diretamente no *OMNIVERSE*, permitindo simulações de física estáveis, realistas e customizáveis, tanto para corpos volumétricos

quanto para tecidos.

Personalização via scripts: O *OMNIVERSE* permite a criação de *scripts* personalizados utilizando C++ ou *python* utilizando *plugins*.

3.5 DISCUSSÃO FINAL

A Tabela 1 apresenta um sumário dos principais recursos disponíveis e apresentados neste capítulo sobre *soft body* e *cloth* para modelagem e simulação de objetos deformáveis na *GODOT*, *UNITY*, *UNREAL ENGINE* e *OMNIVERSE*:

Tabela 1 – Comparação entre recursos de *soft body* e *cloth* nas principais engines de jogos

Recurso	Godot 4 (SoftBody3D)	Unity 6 (Cloth / Obi Soft- Body)	Unreal Engine 5 (Chaos Flesh / Cloth)	OMNIVERSE PHYSX
Suporte Nativo	Sim, com <i>SoftBody3D</i>	Parcial: Cloth nativo; <i>Obi SoftBody</i> via <i>asset store</i>	Sim, via <i>Chaos Flesh</i> e <i>Chaos Cloth</i>	Sim, utilizando a <i>API</i> do <i>PhysX</i> ou utili- zando o <i>plugin Phy-</i> <i>sicsBundle</i>
Estabilidade	Instável; diversos <i>bugs</i> conhecidos	Estável com <i>Cloth</i> ; avançado e estável com <i>Obi</i>	Alta estabilidade; pro- jetado para produção	Alta estabilidade; pro- jetado para produção
Controle Físico	Massa total, coefi- ciente de pressão, ri- gidez linear, coefi- ciente de amorteci- mento, pontos fixos	Rigidez de alonga- mento e de flexão, amortecimento, coli- são contínua, partícu- las virtuais	Densidade, rigidez de vértice, amorte- cimento de vértice, incompressibilidade (volume), atrito, arrasto/vento	Densidade, Atrito Dinâmico, Módulo de Young, Coefi- ciente de Poisson, Amortecimento de Elasticidade, Escala de Amortecimento
Integração com Fí- sica	Interage com corpos rígidos; pode usar co- lisores	Colisão com esferas e cápsulas; limitado na- tivamente	Total integração com sistema físico e de ani- mação	Total integração com o sistema físico e de animação
Personalização via Código	Sim, com <i>GDScript</i> ou C#	Sim, com C#; <i>Obi</i> in- clui sistema de even- tos	Sim, com <i>Blueprints</i> e C++	Sim, com C++ ou <i>Python</i> utilizando <i>plu-</i> <i>gins</i>
Ferramentas Visuais	Limitadas; maioria via <i>script</i>	Ferramentas básicas no editor, avançadas no <i>Obi</i>	Edição no <i>viewport</i> , pintura de pesos (ou <i>weight painting</i>), com- patível com <i>Sequen-</i> <i>cer</i>	Limitadas; maioria via <i>script</i>

4 TRABALHOS RELACIONADOS

Este capítulo apresenta uma análise comparativa dos principais trabalhos sobre modelagem e simulação de tecidos e corpos deformáveis aplicados a ferramentas gráficas computacionais e motores de jogos que avançaram o estado da arte. Os métodos foram agrupados em três categorias principais: abordagens geométricas, métodos físicos e híbridos, e *frameworks* baseados em elementos finitos e partículas. A última seção inclui uma tabela comparativa com as vantagens e limitações de cada proposta.

4.1 ABORDAGENS GEOMÉTRICAS

Weil (WEIL, 1986) foi um dos pioneiros ao utilizar apenas geometria para visualizar tecidos. Ele representou o pano como uma malha de pontos conectados por curvas de catenária, cuja forma natural simula o caimento de cabos pesados. As interseções entre curvas eram resolvidas removendo-se a inferior, criando um esqueleto curvo. Triângulos formados pelas curvas eram subdivididos e suavizados com curvas em “S” do tipo *splines* para gerar a superfície, tratada como objeto cilíndrico em *ray tracing* (WEIL, 1986). Embora rápido, o método só simula tecido pendurado e ignora propriedades físicas.

Agui *et al.* (AGUI; NAGAO; NAKAJMA, 1990) propuseram um modelo geométrico para mangas de camisa dobradas, baseando-se na diferença de curvatura entre as partes interna e externa da manga. Usaram anéis circulares convertidos em polígonos com vincos definidos por mínimas variações de distância ao corpo. Para suavizar ângulos, aplicaram média entre vértices vizinhos. Apesar de eficaz para mangas, o método não detalha critérios para identificação de dobras.

Ng *et al.* (NG; GRIMSDALE; ALLEN, 1995) desenvolveram o editor *Geoff*, que associa camadas de tecido à pele subjacente e traça linhas de dobra baseadas em folgas. Aplicaram funções senoidais nessas linhas e permitiram edição manual. O método é rápido e adequado para ferramentas de animação interativa.

4.2 MODELOS FÍSICOS BASEADOS EM MALHAS

Feynman (FEYNMAN, 1986) propôs modelar tecidos como uma malha 2D de pontos conectados, baseando-se na teoria de placas elásticas. Derivou uma função de energia combinando termos de elasticidade, curvatura e gravidade, e utilizou gradiente descendente com

aceleração *multigrid*. Embora eficiente, não reporta tempos de execução.

Aono (AONO, 1990) usou a teoria da elasticidade linear para descrever propagação de rugas como perturbações dinâmicas. Derivou uma equação diferencial com inércia, rigidez e amortecimento viscoso. Focou apenas em rugas, sem considerar interação com objetos.

O grupo da Universidade de Quioto (SAKAGUCHI; MINOH; IKEDA, 1991) apresentou o sistema *Party17*, onde o tecido é modelado por pontos de massa interligados por molas com forças elásticas, viscosas e plásticas. Utilizaram o método de Euler para integração temporal e volumes limitadores para colisões, garantindo realismo com atrito e contenção.

Provot (PROVOT *et al.*, 1995) usou o modelo clássico de massas e molas, adicionando um limite de alongamento para evitar buracos. Implementou uma simulação eficiente com método de Euler, mostrando bons resultados para tecidos pendurados e bandeiras ao vento.

4.3 MODELOS HÍBRIDOS E ELEMENTOS FINITOS

Hinds *et al.* (NG; GRIMSDALE, 1996) propuseram um sistema de design de roupas baseado em painéis 3D sobre corpos digitalizados. Utilizaram interpolação para gerar continuidade entre painéis e harmônicas para adicionar dobras. Em trabalhos posteriores, transformaram os painéis em moldes 2D.

Okabe *et al.* (OKABE *et al.*, 1992) automatizaram a adaptação de moldes 2D em corpos 3D com teoria de elasticidade de cascas. Aplicaram gradiente descendente adaptativo para encontrar a forma de equilíbrio do tecido, simulando caimento em cerca de 20 minutos.

Kunii *et al.* (NG; GRIMSDALE, 1996) propuseram um modelo híbrido: simularam o tecido com energia métrica e de curvatura, extraíram contornos de rugas e os transformaram em curvas senoidais. Aplicaram uma nova minimização para gerar rugas realistas.

4.4 MODELAGEM COM SPH E CFD

Smoothed Particle Hydrodynamics (SPH) ou Hidrodinâmica de Partículas Suavizadas e *Computational Fluid Dynamics* (CFD) ou Dinâmica dos Fluidos Computacional são duas técnicas de modelagem de fluidos aplicáveis à modelagem de tecidos deformáveis.

Li *et al.* (NG; GRIMSDALE, 1996) modelaram a interação de tecidos com vento aplicando equações médias de Navier-Stokes sobre *patches* de tecido. Usaram método de Terzopoulos para deformação, apresentando animações realistas de bandeiras ao vento.

Harada (HARADA; KOSHIZUKA; KAWAGUCHI, 2007) aplicou o método SPH

para simular líquidos com partículas interativas. Utilizou estruturas de vizinhança baseadas em baldes e implementações na GPU para ganho de desempenho, simulando em tempo real.

4.5 CONSIDERAÇÕES FINAIS

Os trabalhos revisados exploram diferentes estratégias para modelar tecidos e corpos deformáveis, variando de métodos geométricos simples a simulações físicas complexas. As abordagens mais modernas combinam precisão física com eficiência computacional, utilizando estruturas de dados otimizadas e suporte de *hardware* gráfico. A Tabela 2 resume as principais vantagens e desvantagens de cada trabalho citado.

Tabela 2 – Comparativo entre os principais trabalhos analisados

Autor(es)	Vantagens	Desvantagens
Weil (1986)	Simples e eficiente; sem cálculo físico	Só para tecido pendurado; ignora propriedades físicas
Agui et al. (1990)	Modela dobras com simetria e suavização	Aplicável apenas a mangas dobradas
Ng et al. (1996)	Interativo e personalizável	Falta realismo físico
Feynman (1996)	Boa modelagem física; multi-grid acelera	Tempo de execução não informado
Aono (1996)	Analisa propagação de rugas	Ignora interação com objetos
Kyoto Univ. (1996)	Realista e com colisão com atrito	Tempo computacional elevado
Provot (1995)	Simples, eficaz, com controle de alongamento	Sem colisões entre elementos do tecido
Hinds et al. (1996)	Permite design realista e fabricação	Exige entrada manual precisa
Okabe et al. (1996)	Integra modelagem e caimento realista	Simulação ainda lenta
Kunii et al. (1996)	Rugas visuais com base física inicial	Foco limitado em pontos característicos
Li et al. (1996)	Interação com vento realista	Alto custo computacional com fluxo instável
Harada (2007)	Simulação em tempo real via GPU	Exige hardware gráfico moderno

5 METODOLOGIA

Esta pesquisa caracteriza-se por ser de natureza qualitativa, com objetivos exploratórios e descritivos. Para a sua realização, foram empregados os procedimentos técnicos de pesquisa bibliográfica e pesquisa documental.

A fundamentação teórica sobre as técnicas de modelagem de tecidos foi desenvolvida a partir da análise de um artigo científico do tipo análise sistemática (NG; GRIMSDALE, 1996). Esse artigo apresenta uma visão abrangente sobre o tema descrevendo 19 técnicas distintas. Adicionalmente, foram investigadas 4 técnicas de modelagem de corpos deformáveis por meio dos artigos massa-mola (PROVOT *et al.*, 1995), massa-mola-amortecedor (NIKOOYAN; ZADPOOR, 2011), método dos elementos finitos (DEBUNNE *et al.*, 2000) e métodos baseados em partículas (HARADA; KOSHIZUKA; KAWAGUCHI, 2007). As técnicas foram selecionadas com base na sua relevância histórica e impacto no campo da Computação Gráfica e de aplicação gráficas interativas.

Para a fundamentação tecnológica, foram selecionadas três *game engines* proeminentes no mercado: *Godot 4* (GODOT, 2025), *Unity 6* (UNITY-TECHNOLOGIES, 2025) e *Unreal Engine 5* (EPIC-GAMES, 2025). A elas, somou-se a plataforma de simulação e colaboração *NVIDIA Omniverse* (NVIDIA, 2025b). A escolha destas quatro plataformas foi direcionada pelos requisitos de relevância, ampla utilização na indústria (de jogos e simulação) e disponibilidade de documentação técnica.

O estudo da implementação de *soft bodies* em cada uma das plataformas foi realizado por meio de pesquisa documental, analisando-se as documentações oficiais fornecidas pelos seus respectivos desenvolvedores. Complementarmente, foram realizadas pesquisas em vídeos no *YouTube* e em fóruns especializados (como o da *Unreal engine*). A análise focou em identificar a facilidade de implementação, a curva de aprendizado e o gasto computacional de cada solução.

As informações coletadas na fundamentação teórica e tecnológica foram estruturadas e analisadas criticamente, buscando-se estudar e assimilar cada uma das técnicas, bem como compreender como a simulação de *soft bodies* evoluiu ao longo dos anos, além de identificar paralelos entre os modelos teóricos e as implementações práticas.

Para a fase experimental do trabalho, foram desenvolvidos dois casos de estudo práticos, implementados de forma comparativa nas quatro plataformas tecnológicas selecionadas: *Godot*, *Unity*, *Unreal Engine* e *NVIDIA Omniverse*. O objetivo foi analisar os procedimentos de implementação, a curva de aprendizado e as ferramentas disponíveis para a simulação de corpos

deformáveis (*soft bodies*) e tecidos (*cloth*) em cada ambiente.

Os casos de estudo foram:

- **Deformação e Acoplamento de Esferas:** Este teste consistiu em simular uma esfera deformável colidindo e se acoplando sobre uma esfera rígida.
 - Na *Godot*, utilizou-se o nó *SoftBody3D*, explorando parâmetros como o *Pressure Coefficient* para alterar o comportamento da colisão.
 - Na *Unity*, a simulação foi implementada aplicando o componente nativo *Cloth* a uma malha esférica.
 - Na *Unreal Engine*, o processo envolveu a conversão de uma malha estática em um tetraedro, utilizando o solver *Chaos Flesh* e um *Deformable Collisions Actor*.
 - Na *NVIDIA Omniverse*, a cena foi configurada com uma esfera *Deformable Body* (baseada em FEM) e um *Physics Material* para definir sua elasticidade, colidindo contra uma esfera rígida padrão com um componente *Collider*.
- **Simulação de Toalha sobre uma Mesa:** Este teste avaliou a simulação de tecido (*cloth*), observando a aderência de uma malha plana sobre a superfície de uma mesa circular.
 - Na *Godot*, aplicou-se o nó *SoftBody3D* a uma malha plana (*quad*) com um número elevado de subdivisões para simular a fluidez do tecido.
 - Na *Unity*, buscou-se utilizar o componente *Cloth*. A metodologia considerou a limitação documentada deste componente, que nativamente só interage com colisores do tipo esfera e cápsula, analisando a viabilidade de sua aplicação em uma geometria de mesa.
 - Na *Unreal Engine*, uma malha retangular foi convertida em uma *Skeletal Mesh* para que pudesse receber a propriedade *Cloth* do sistema *Chaos Cloth*, permitindo o ajuste de parâmetros físicos detalhados, como densidade e rigidez.
 - Na *NVIDIA Omniverse*, foi necessário gerar uma malha plana customizada (50x50) via *script Python*, pois a primitiva padrão não era adequada. A essa malha foi aplicada a propriedade *Particle Cloth*.

A experiência de executar esses dois testes foi essencial para o aprendizado. Esses casos proporcionaram uma visão geral clara sobre a criação, implementação e configuração de corpos e tecidos deformáveis nas principais ferramentas do mercado.

Isso permitiu:

- Analisar o estado da arte atual e o avanço da área ao longo dos anos.

- Avaliar a qualidade de cada ferramenta ao lidar com objetos complexos.

6 CASOS EXPERIMENTAIS

Neste capítulo, será apresentada uma análise aprofundada sobre os conhecimentos adquiridos a respeito de simulações com *soft bodies* (corpos deformáveis), com foco em sua implementação e comportamento em três das principais *engines* de desenvolvimento de jogos: *Godot*, *Unity* e *Unreal Engine*, além de uma análise correspondente na plataforma de simulação *NVIDIA Omniverse*, que utiliza o motor de física *PhysX* em seu núcleo. Para isso, foram desenvolvidas e testadas duas simulações distintas:

- **Deformação e acoplamento entre esferas**, onde uma esfera se deforma e se encaixa em outra, representando interações físicas complexas entre objetos.
- **Aderência de uma malha simulando uma toalha sobre uma mesa**, com o objetivo de observar como cada plataforma lida com o comportamento físico de tecidos e superfícies flexíveis.

Além de descrever os resultados visuais e qualitativos dessas simulações, também foram coletadas **métricas de desempenho computacional**, visando compreender o custo de processamento associado a cada abordagem. Outro ponto analisado foi a **facilidade de implementação**, considerando tanto a curva de aprendizado quanto a praticidade das ferramentas e recursos oferecidos por cada plataforma para modelagem e simulação de *soft bodies*.

Para garantir a consistência dos dados, todos os experimentos foram executados em uma estação de trabalho equipada com um processador **Intel Core i5-12400F** e uma placa de vídeo **NVIDIA GeForce RTX 3060 (12GB)**. É importante ressaltar que os valores de desempenho apresentados nas tabelas deste capítulo correspondem a medições isoladas. Foi realizado um monitoramento durante a execução das simulações, e os dados expostos refletem a **média aritmética** durante a execução.

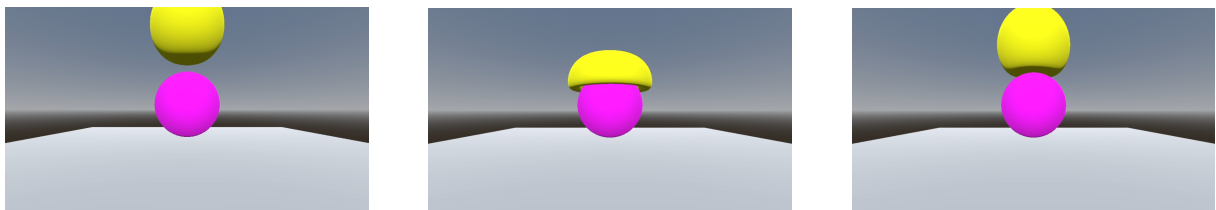
Essa análise comparativa busca oferecer uma visão clara das vantagens e limitações de cada ambiente no contexto da simulação de corpos deformáveis, contribuindo para decisões mais informadas no desenvolvimento de aplicações interativas e jogos digitais que envolvam física avançada.

6.1 GODOT

6.1.1 Simulação de acoplamento entre esferas

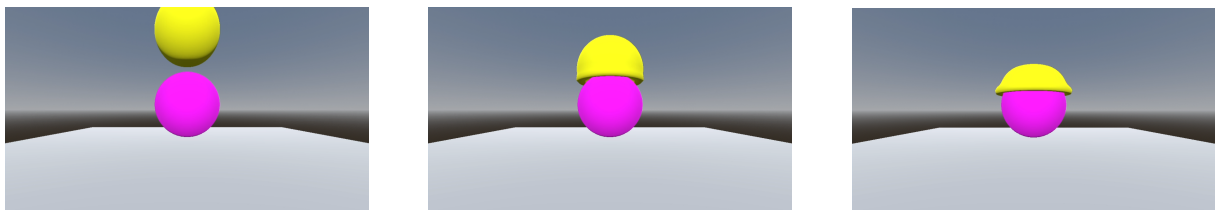
No Godot 4, utilizando o *Jolt Physics*, é possível implementar a interação de acoplamento entre esferas por meio de duas abordagens distintas. Na primeira abordagem, o aumento do parâmetro *Pressure Coefficient* provoca uma deformação considerável da esfera, permitindo que ela se acople à outra; contudo, este ajuste gera um efeito de *bouncing*, que ocasiona a reconstrução da forma da esfera e a subsequente elevação (salto). Na segunda abordagem, a redução do *Pressure Coefficient* resulta em uma colisão entre as esferas, fazendo com que uma delas se fixe à outra de maneira semelhante a um chapéu, acompanhando o contorno da esfera base.

Figura 2 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera totalmente deformada; e na terceira, já restituída, prestes a pular.



Fonte: Criada pelo autor.

Figura 3 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera parcialmente deformada; e na terceira, já completamente deformada e acoplada a outra esfera.



Fonte: Criada pelo autor.

6.1.2 Simulação da toalha sobre uma mesa

Na simulação da toalha de mesa, também empregando o *Jolt Physics*, o incremento no número de subdivisões da malha *quad* proporciona um efeito fluido, permitindo que a toalha se adapte com precisão ao contorno da mesa. Contudo, em virtude de desafios relacionados à

definição das colisões, verifica-se que a toalha mantém o movimento mesmo após o decurso do tempo.

Figura 4 – Na primeira imagem, o instante antes da colisão; na segunda, a toalha parcialmente deformada; e na terceira, já completamente deformada e acoplada na mesa.



Fonte: Criada pelo autor.

6.1.3 Medidas de desempenho

Tabela 3 – Média de desempenho das simulações na Godot

Caso	Tempo por quadro (ms)	CPU (ms)	GPU (ms)
Esfera 1	1,48	0,11	0,22
Esfera 2	1,59	0,12	0,20
Toalha	1,66	0,12	0,20

As métricas de desempenho foram aferidas na *Godot* utilizando o *plugin Debug Menu* (CALINO, 2023), obtido da *Godot Asset Library*. Os valores médios indicam que todas as simulações são extremamente otimizadas e de baixo custo computacional.

Com tempos médios para produzir um quadro variando minimamente entre 1,48 ms (“Caso Esfera 1”), 1,59 ms (“Caso Esfera 2”) e 1,66 ms (“Caso Toalha”), a carga de processamento é insignificante para a máquina de testes. Os tempos de CPU mantiveram-se em 0,11 ms e 0,12 ms, e os de GPU entre 0,20 ms e 0,22 ms.

Tecnicamente, a GPU representa o gargalo da execução em todos os cenários, pois seu tempo de processamento é consistentemente superior ao da CPU. No entanto, o desempenho geral é tão elevado que essa distinção se torna irrelevante na prática, com todas as simulações operando com uma carga de processamento residual.

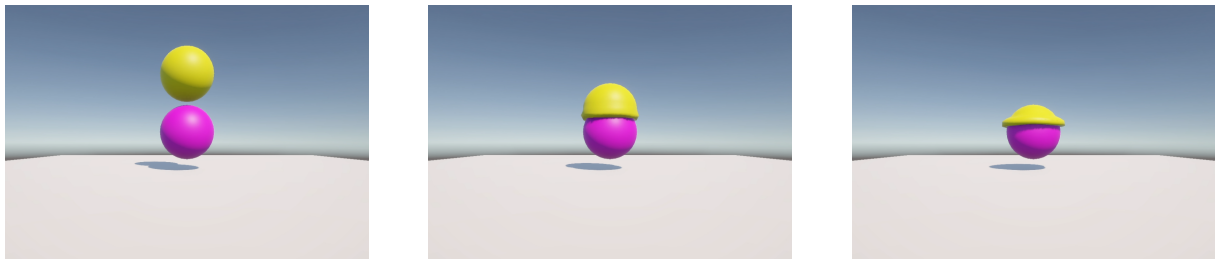
6.2 UNITY

6.2.1 Simulação de acoplamento entre esferas

Na Unity6 é possível criar essa simulação adicionando uma propriedade *Cloth* a um objeto 3D *Sphere* e modificando suas propriedades. Dessa forma, a esfera deformável se acopla perfeitamente à outra esfera rígida, produzindo o efeito de conformar seu formato aos contornos da esfera inferior. As colisões são gerenciadas com precisão, fazendo com que a esfera se comporte como um chapéu sobre a outra, permanecendo completamente estacionária.

A execução do Caso 2 é inviável utilizando recursos nativos do Unity. O componente *Cloth* foi projetado exclusivamente para a simulação de tecidos, não atuando como um solver de *soft-body* de propósito geral. Ele carece dos parâmetros de elasticidade e preservação de volume necessários para simular um corpo sólido que se deforma e retorna à sua forma original, impedindo a criação de um efeito de *bouncing* realista.

Figura 5 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera parcialmente deformada; e na terceira, já completamente deformada e acoplada a outra esfera.



Fonte: Criada pelo autor.

6.2.2 Simulação da toalha sobre uma mesa

Devido ao fato de que o componente *Cloth* possui colisão apenas com esferas e cápsulas, torna-se inviável realizar essa simulação utilizando exclusivamente os recursos oferecidos pela Unity. Isso se dá porque não é possível encaixar nem uma esfera nem uma cápsula em uma mesa plana e circular, uma vez que tais elementos formam uma malha convexa.

6.2.3 Medidas de desempenho

As métricas de desempenho foram aferidas utilizando o *Unity Profiler*. Os valores médios para o cenário (Caso 1) indicam uma simulação extremamente otimizada e de baixo

Tabela 4 – Média de desempenho das simulações na Unity

Caso	Tempo por quadro (ms)	CPU (ms)	GPU (ms)
Esfera 1	1,30	1,28	0,68
Esfera 2	×	×	×
Toalha	×	×	×

custo computacional.

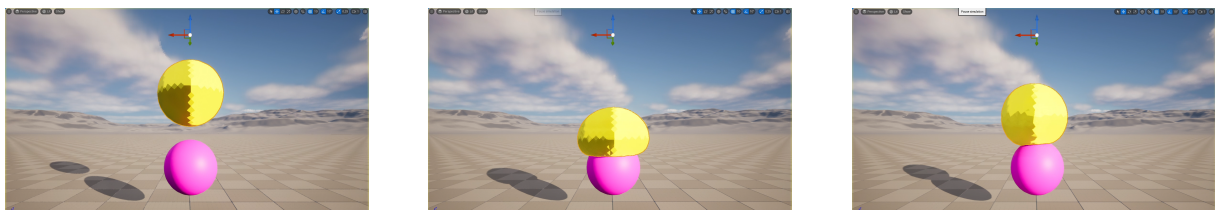
Com um tempo médio de CPU de 1,28 ms e um tempo de GPU de 0,68 ms, a carga de processamento é mínima para ambos os componentes. Tecnicamente, a CPU ainda representa o gargalo da execução, mas o desempenho geral é tão alto que isso se torna irrelevante. A simulação opera com uma carga de processamento muito baixa.

6.3 UNREAL ENGINE

6.3.1 Simulação de acoplamento entre esferas

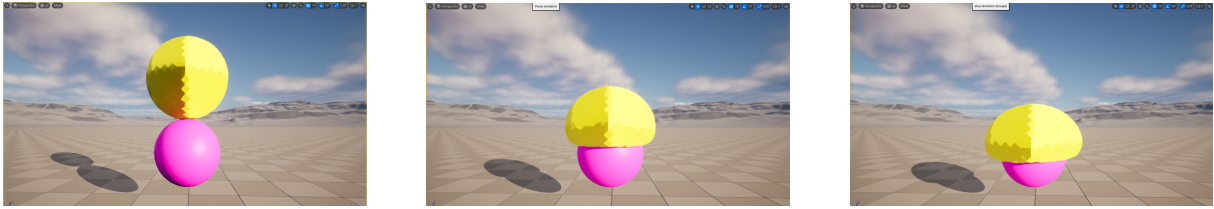
Na *Unreal Engine*, iniciamos criando uma malha estática esférica e a convertemos em um tetraedro. Em seguida, adicionamos o solver de física à cena e incluímos o *Deformable Collisions Actor*, o que permite que as esferas se acoplem perfeitamente, mantendo a integridade estrutural em cada ponto de contato.

Figura 6 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera totalmente deformada; e na terceira, já restituída, prestes a pular.



Fonte: Criada pelo autor.

Figura 7 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera parcialmente deformada; e na terceira, já completamente deformada e acoplada a outra esfera.



Fonte: Criada pelo autor.

6.3.2 Simulação da toalha sobre uma mesa

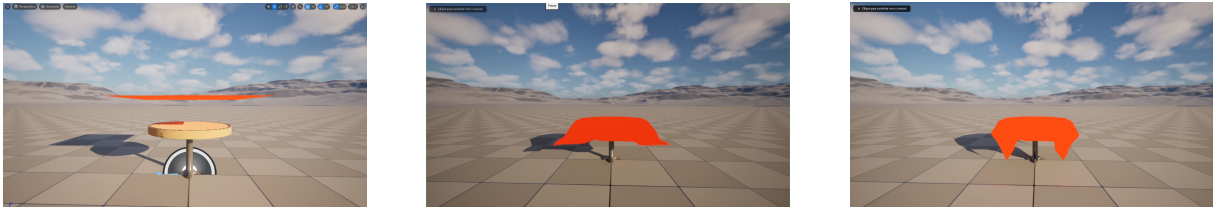
Para simular a toalha de mesa, começamos criando uma malha estática em formato retangular e, em seguida, a convertimos em uma malha esquelética capaz de receber a propriedade *Cloth*. A partir daí, é possível ajustar parâmetros para reproduzir diferentes tipos de tecido. Neste caso, escolhemos simular seda (*silk*), configurando as seguintes propriedades:

- *Density* = 0,1
- *Bend Stiffness* = 0,76
- *Area Stiffness* = 0,1
- *Tether Stiffness* = 0,85
- *Tether Scale* = 1,025
- *Friction* = 0,8
- *Damping* = 0,025
- *Drag* = 0,3
- *Lift* = 0,09
- *Gravity Scale* = 0,9

Combinadas às físicas avançadas do *Chaos Cloth*, essas definições resultam em um movimento suave, leve e realista, capturando tanto a fluidez característica da seda quanto sua resposta a forças como vento e gravidade.

6.3.3 Medidas de desempenho

Figura 8 – Na primeira imagem, o instante antes da colisão; na segunda, a toalha parcialmente deformada; e na terceira, já completamente deformada e acoplada na mesa.



Fonte: Criada pelo autor.

Tabela 5 – Média de desempenho das simulações na Unreal

Caso	Tempo por quadro (ms)	CPU (ms)	GPU (ms)
Esfera 1	24,54	24,86	10,52
Esfera 2	29,69	23,04	10,49
Toalha	16,80	16,82	8,54

As métricas de desempenho foram aferidas utilizando as ferramentas de *profiling* da *Unreal Engine 5*. A análise distingue a eficiência média entre os sistemas *Chaos Cloth* e *Chaos Flesh*.

A simulação com *Chaos Cloth* (“Caso Toalha”) demonstrou ser a mais otimizada, com um tempo médio de quadro de 16,80 ms. A carga computacional média se dividiu em 16,82 ms para a CPU e 8,54 ms para a GPU. Em contrapartida, as simulações utilizando *Chaos Flesh* (“Caso Esfera 1” e “Caso Esfera 2”) apresentaram um custo significativamente maior. O cenário mais pesado, “Caso Esfera 2”, exigiu uma média de 29,69 ms por quadro, com 23,04 ms de processamento na CPU.

Fica evidente que, nestes testes, o sistema *Chaos Flesh* é computacionalmente mais intensivo que o *Chaos Cloth*. Em todos os cenários, a CPU foi o gargalo de performance, indicando que a complexidade das simulações de física impacta primariamente o processador i5-12400F utilizado.

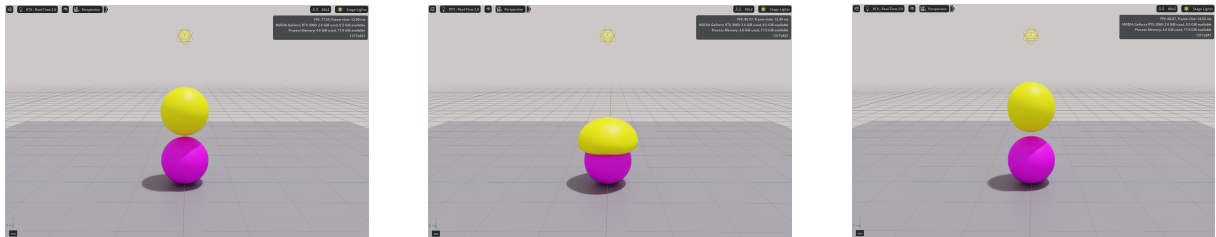
6.4 OMNIVERSE

6.4.1 Simulação de acoplamento entre esferas

A implementação da simulação no *Omniverse* emprega duas malhas esféricas (*sphere mesh*). A primeira é definida como um corpo rígido com a propriedade *Collider*. A segunda é um *Deformable Body*, à qual se associa um *Physics Material*. Este material é essencial para

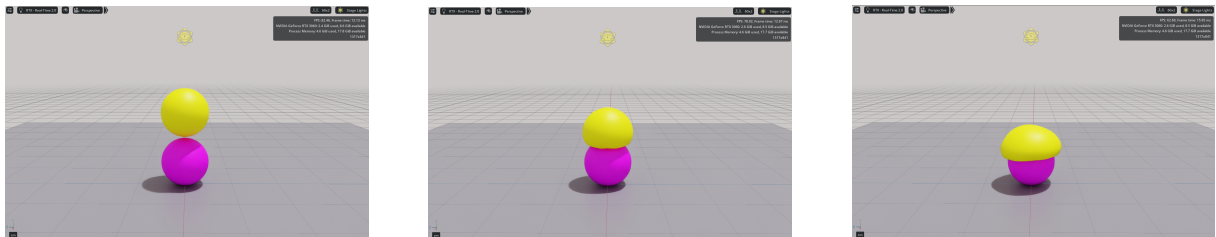
definir os parâmetros físicos do corpo deformável, como o *Módulo de Young*. O sistema de física gerencia a interação entre ambas as formas de maneira estável e sem apresentar falhas de colisão.

Figura 9 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera totalmente deformada; e na terceira, já restituída, prestes a pular.



Fonte: Criada pelo autor.

Figura 10 – Na primeira imagem, o instante antes da colisão; na segunda, a esfera parcialmente deformada; e na terceira, já completamente deformada e acoplada a outra esfera.



Fonte: Criada pelo autor.

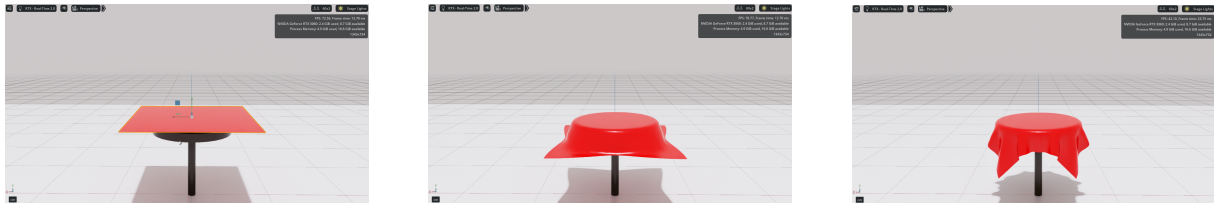
6.4.2 Simulação da toalha sobre a mesa

A simulação de uma toalha sobre a mesa no *Omniverse* requer uma abordagem customizada, visto que a primitiva *Mesh plane* do *USD* não é adequada para essa finalidade. Portanto, uma malha (*mesh*) de 50x50 é gerada dinamicamente utilizando *Python* (D). Subsequentemente, a propriedade *Particle Cloth* é aplicada a esta malha e configurada. A mesa, por sua vez, recebe o componente *Collider* para habilitar a detecção de colisão.

6.4.3 Medidas de desempenho

O primeiro caso teve um tempo médio de *frame* de 25.64 ms, sendo este o cenário mais lento. O alto tempo médio de *CPU* (15.38 ms) sugere que a complexidade do cálculo de colisão, deformação e posterior restauração (o efeito *bouncing* e reconstrução) do *soft body* é o fator mais exigente. O segundo caso, teve o tempo médio de *CPU* ligeiramente inferior (15.18

Figura 11 – Na primeira imagem, o instante antes da colisão; na segunda, a toalha parcialmente deformada; e na terceira, já completamente deformada e acoplada na mesa.



Fonte: Criada pelo autor.

Tabela 6 – Média de desempenho das simulações no Omniverse

Caso	Tempo por quadro (ms)	CPU (ms)	GPU (ms)
Esfera 1	25,64	15,38	10,26
Esfera 2	25,31	15,18	10,12
Toalha	23,36	14,02	9,34

ms), o que indica que o comportamento de acoplar-se na outra esfera é um cálculo de física um pouco menos dispendioso do que a deformação elástica e o *bouncing* do Caso 1, mas ainda assim é uma simulação complexa. E por último, o caso da toalha, foi o com o melhor tempo médio de frame de 23.36 ms, sendo a simulação mais eficiente. O menor tempo de *CPU* (14.02 ms) confirma que a física de simulação de tecidos (*cloth simulation*) e seu acoplamento à mesa é significativamente menos pesada para a *CPU* do que as interações e deformações do tipo *soft body* dos Casos 1 e 2.

Em suma, o uso do *DLSS Performance* garante que a *GPU RTX 3060* não seja o gargalo (≈ 10 ms), mas a performance geral (39 a 43 *FPS*) é ditada pela *CPU*, que passa a maior parte do tempo processando os cálculos de física. Quanto mais complexa e detalhada a interação do *soft body*, maior o tempo de *CPU* e menor o *FPS*.

6.5 DISCUSSÃO FINAL

A análise comparativa dos testes e resultados revela um claro panorama de trade-offs entre as plataformas:

- **Godot:** Apresenta a implementação mais leve e prática, facilitando o desenvolvimento inicial. No entanto, demonstrou ser também a solução mais instável durante os testes.
- **Unity:** Obteve um bom desempenho no “caso 1”, mas revelou limitações críticas. A incapacidade do seu sistema nativo de simulação (de tecidos/corpos deformáveis) de colidir com geometrias complexas, restringindo-se a esferas e cilindros, torna a ferramenta

inviável para as simulações propostas sem o uso de *plugins* externos.

- ***Unreal Engine***: Mostrou resultados robustos e alta estabilidade, provando-se uma opção viável. Contudo, seu foco em realismo gráfico de alta fidelidade resulta em um custo computacional extremamente elevado.
- ***NVIDIA Omniverse***: Demonstrou qualidade superior a todas as concorrentes. A plataforma gerenciou de forma excelente as colisões e as três simulações propostas, consolidando-se como uma ferramenta poderosa para produção. Este foi, entretanto, o sistema mais custoso computacionalmente, exigindo a ativação do *DLSS* em modo *Performance* para se obter uma taxa de quadros média satisfatória na configuração de hardware utilizada (RTX 3060).

A escolha da ferramenta depende de um balanço direto entre fidelidade de simulação e custo de *hardware*. Enquanto *Godot* e *Unity* falham em robustez ou recursos nativos, a decisão entre *Unreal Engine* e *Omniverse* é uma escolha entre fidelidade “muito boa” com alto custo (*Unreal*) e fidelidade “superior” com custo extremo (*Omniverse*).

7 CONCLUSÃO E TRABALHOS FUTUROS

Este trabalho de Conclusão de Curso investigou as principais técnicas de modelagem e simulação física de tecidos (*cloth*) e corpos deformáveis (*soft bodies*) em Computação Gráfica, utilizando motores de jogos para o desenvolvimento de animações por computador.

O estudo atingiu seus objetivos ao revisar a fundamentação teórica, desde métodos geométricos pioneiros e abordagens baseadas em física, como sistemas massa-mola e o Método dos Elementos Finitos, até técnicas híbridas e baseadas em partículas para desenvolver casos de estudo no contexto de animações geradas para jogos e simuladores.

O estudo comparativo de técnicas, que envolveram o aprendizado da fundamentação tecnológica, culminou em diferentes implementações de exemplos de objetos em animação do tipo *soft body* e *cloth*, utilizando três motores de jogos bastante populares (*Godot*, *Unity*, e a *Unreal Engine*) e uma ferramenta de simulação da *NVIDIA* chamada *Omniverse*.

A investigação realizada neste trabalho permite concluir que, embora a simulação de corpos e tecidos deformáveis seja uma área de pesquisa estabelecida e essencial para a computação gráfica, ainda permanece um desafio fundamental e um campo ainda com possibilidades de avanço no estado-da-arte.

Apesar da evolução drástica no poder de processamento gráfico e das décadas de investigação que produziram diversas abordagens—desde métodos geométricos e sistemas massa-mola até técnicas complexas como o Método dos Elementos Finitos (FEM) e Métodos Baseados em Partículas (SPH), o estado da arte ainda carece de uma solução definitiva. A vasta gama de propostas de diferentes autores, revisadas nos Capítulos 2 e 4, evidencia uma busca contínua por um método que unifique realismo físico, estabilidade e eficiência computacional, algo que ainda não foi alcançado.

Esta lacuna teórica reflete-se diretamente nas implementações práticas das *game engines* e plataformas de simulação modernas. Mesmo com hardware avançado, os softwares atuais ainda demonstram dificuldades significativas para implementar e executar simulações de corpos deformáveis de forma robusta.

Os casos experimentais deste trabalho (Capítulo 6) comprovaram essa dificuldade: A *Godot* mostrou-se leve, porém instável. A *Unity* revelou limitações críticas em seu sistema nativo, que se mostrou inviável para colisões com geometrias complexas sem o uso de *plugins* externos. A *Unreal Engine* e a *NVIDIA Omniverse*, embora entreguem resultados de alta fidelidade e robustez, o fazem ao custo de um desempenho computacional extremo. Conclui-se, portanto,

que o campo ainda é incipiente em termos de soluções práticas e acessíveis. O claro *trade-off* entre fidelidade e custo de *hardware* demonstra que o problema não é apenas de processamento, mas fundamentalmente algorítmico.

Como trabalhos futuros, o foco será aprofundar esta investigação, explorando abordagens emergentes que tentam contornar essas limitações, como o uso de inteligência artificial para modelagem e otimização, e aplicar os estudos de caso em um contexto mais amplo que envolva aplicações gráficas interativas mais complexas.

REFERÊNCIAS

- AGUI, T.; NAGAO, Y.; NAKAJIMA, M. An expression method of cylindrical cloth objects—an expression of folds of a sleeve using computer graphics. **Trans. of Soc. Of Electronics, Information and Communication, Col. J73-D-II**, v. 7, p. 1095–1097, 1990.
- AONO, M. A wrinkle propagation model for cloth. In: SPRINGER. **CG International'90: Computer Graphics Around the World**. [S.l.], 1990. p. 95–115.
- AUBEL, A.; THALMANN, D. Realistic deformation of human body shapes. In: SPRINGER. **Computer Animation and Simulation 2000: Proceedings of the Eurographics Workshop in Interlaken, Switzerland, August 21–22, 2000**. [S.l.], 2000. p. 125–135.
- BARAFF, D.; WITKIN, A. Large steps in cloth simulation. In: **Seminal Graphics Papers: Pushing the Boundaries, Volume 2**. [S.l.: s.n.], 2023. p. 767–778.
- CALINOU. **Debug Menu - Display in-game FPS/performance/hardware metrics - Godot Asset Library** — [godotengine.org](https://godotengine.org/asset-library/asset/1902). 2023. <<https://godotengine.org/asset-library/asset/1902>>. [Accessed 12-11-2025].
- COUMANS, E. **Bullet Real-Time Physics Simulation | Home of Bullet and PyBullet: physics simulation for games, visual effects, robotics and reinforcement learning**. — pybullet.org. 2025. <<https://pybullet.org/wordpress/>>.
- DEBUNNE, G.; DESBRUN, M.; CANI, M.-P.; BARR, A. Adaptive simulation of soft bodies in real-time. In: IEEE. **Proceedings computer animation 2000**. [S.l.], 2000. p. 15–20.
- EPIC-GAMES. **Unreal Engine 5.5 Documentation | Unreal Engine 5.5 Documentation | Epic Developer Community** — [dev.epicgames.com](https://dev.epicgames.com/documentation/en-us/unreal-engine). 2025. <<https://dev.epicgames.com/documentation/en-us/unreal-engine>>.
- FENG, W.-W.; YU, Y.; KIM, B.-U. A deformation transformer for real-time cloth animation. **ACM transactions on graphics (TOG)**, ACM New York, NY, USA, v. 29, n. 4, p. 1–9, 2010.
- FEYNMAN, C. R. **Modeling the appearance of cloth**. Tese (Doutorado) — Massachusetts Institute of Technology, 1986.
- GODOT. **Godot Docs – 4.4 branch** — [docs.godotengine.org](https://docs.godotengine.org/en/stable/). 2025. <<https://docs.godotengine.org/en/stable/>>.
- GOLDENTHAL, R.; HARMON, D.; FATTAL, R.; BERCOVIER, M.; GRINSPUN, E. Efficient simulation of inextensible cloth. In: **ACM SIGGRAPH 2007 papers**. [S.l.: s.n.], 2007. p. 49–es.
- HARADA, T.; KOSHIZUKA, S.; KAWAGUCHI, Y. Smoothed particle hydrodynamics on gpus. In: SBC PETROPOLIS. **Computer Graphics International**. [S.l.], 2007. v. 40, p. 63–70.
- KIM, Y. Design and performance evaluation of soft 3d models using metaball in unreal engine 4. **Journal of Mobile Multimedia**, p. 1795–1810, 2022.
- NG, H. N.; GRIMSDALE, R. L. Computer graphics techniques for modeling cloth. **IEEE Computer Graphics and Applications**, IEEE, v. 16, n. 5, p. 28–41, 1996.
- NG, H. N.; GRIMSDALE, R. L.; ALLEN, W. G. A system for modelling and visualization of cloth material. **Computers & graphics**, Elsevier, v. 19, n. 3, p. 423–430, 1995.

NIKOOYAN, A. A.; ZADPOOR, A. A. Mass–spring–damper modelling of the human body to study running and hopping—an overview. **Proceedings of the institution of mechanical engineers, Part H: Journal of engineering in medicine**, SAGE Publications Sage UK: London, England, v. 225, n. 12, p. 1121–1135, 2011.

NVIDIA. **GitHub - NVIDIA-Omniverse/kit-app-template: Omniverse Kit App Template — github.com**. 2025. <<https://github.com/NVIDIA-Omniverse/kit-app-template?ncid=so-oth-199314>>. [Accessed 30-09-2025].

NVIDIA. **NVIDIA Omniverse — docs.nvidia.com**. 2025. <<https://docs.nvidia.com/omniverse/index.html>>.

OKABE, H.; IMAOKA, H.; TOMIHA, T.; NIWAYA, H. Three dimensional apparel cad system. In: **Proceedings of the 19th annual conference on Computer graphics and interactive techniques**. [S.l.: s.n.], 1992. p. 105–110.

PROVOT, X. *et al.* Deformation constraints in a mass-spring model to describe rigid cloth behaviour. In: CANADIAN INFORMATION PROCESSING SOCIETY. **Graphics interface**. [S.l.], 1995. p. 147–147.

SAKAGUCHI, Y.; MINOH, M.; IKEDA, K. Party: Physical environment of artificial reality for dress simulation (i)—a dynamically deformable model of dress. **Trans. Soc. of Electronics, Information and Communications: Graphics and CAD**, v. 54, p. 25–32, 1991.

UNITY-TECHNOLOGIES. **Unity - Manual: Unity 6.1 User Manual — docs.unity3d.com**. 2025. <<https://docs.unity3d.com/Manual/index.html>>.

WEIL, J. The synthesis of cloth objects. **ACM Siggraph Computer Graphics**, ACM New York, NY, USA, v. 20, n. 4, p. 49–54, 1986.

ANEXOS

ANEXO A – CÓDIGO DE EXEMPLO DO SOFTBODY NA GODOT

```
1 extends SoftBody3D
2
3 @onready var softbody: SoftBody3D = $"."
4
5 func _ready() -> void:
6     softbody.simulation_precision = 50.0
7     softbody.total_mass = 1.0
8     softbody.linear_stiffness = 0.1
9     softbody.pressure_coefficient = 20.0
10    softbody.damping_coefficient = 0.01
11    softbody.ray_pickable = true
```

ANEXO B – CÓDIGO DE EXEMPLO DO SOFTBODY NA UNITY

```
1  using UnityEngine;
2
3  public class ClothConfigurator : MonoBehaviour
4  {
5      void Start()
6      {
7          Cloth clothComponent = GetComponent<Cloth>();
8          if(clothComponent != null)
9          {
10             clothComponent.stretchingStiffness = 0.95f;
11             clothComponent.bendingStiffness = 0.95f;
12             clothComponent.useGravity = true;
13             clothComponent.useTethers = true;
14             clothComponent.damping = 0.05f;
15             clothComponent.friction = 0.5f;
16             clothComponent.collisionMassScale = 2f;
17         }
18         else
19         {
20             Debug.LogWarning("O componente Cloth não foi encontrado neste
                ↪ GameObject.");
21         }
22     }
23 }
```

ANEXO C – CÓDIGO DE EXEMPLO DO SOFTBODY NA UNREAL ENGINE

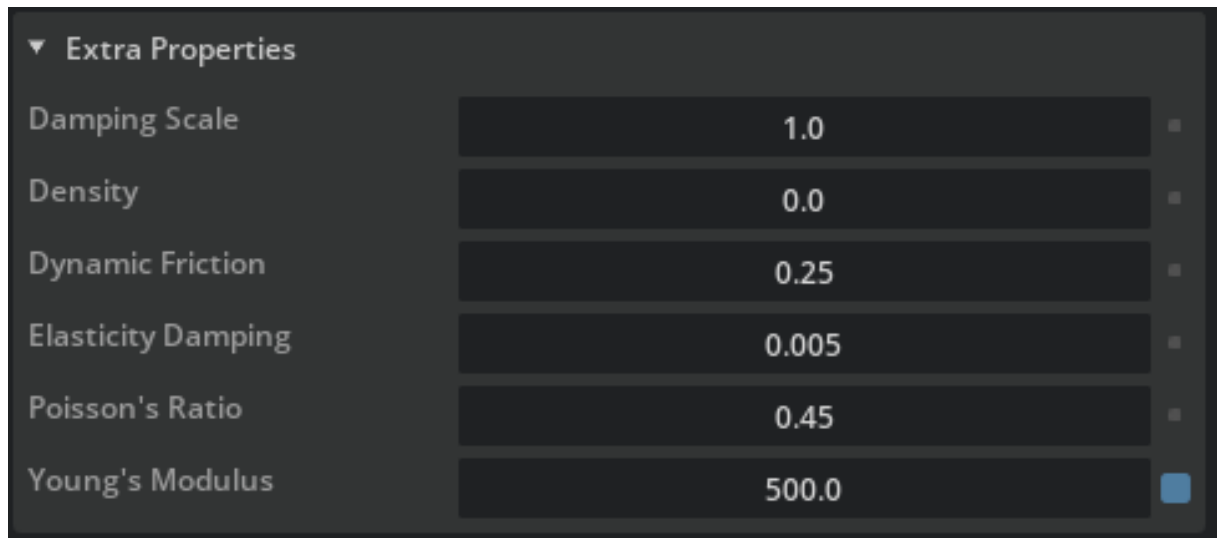
Figura 12 – Fluxo do *Blueprint* para a simulação de corpos deformáveis.



Fonte: Criada pelo autor.

ANEXO D – CÓDIGO PARA GERAR A PLANE MESH NA OMNIVERSE

```
1
2  import omni.kit.commands
3  from pxr import UsdGeom, Gf
4
5  prim_path = "/World/HighResPlane"
6
7  omni.kit.commands.execute("CreateMeshPrimCommand",
8      prim_type="Plane",
9      prim_path=prim_path,
10     u_patches=50,
11     v_patches=50,
12     half_scale=50.0
13 )
14
15 omni.kit.commands.execute("CreateParticleClothCommand",
16     prim_path=prim_path,
17     compute_springs_enabled=True # Opcional: Garante que as molas do
    ⇨ tecido sejam calculadas
18 )
19
```

ANEXO E – CÓDIGO DE EXEMPLO DE CORPO DEFORMÁVEL OMNIVERSE**Figura 13 – Propriedades do corpo deformável no OMNIVERSE**

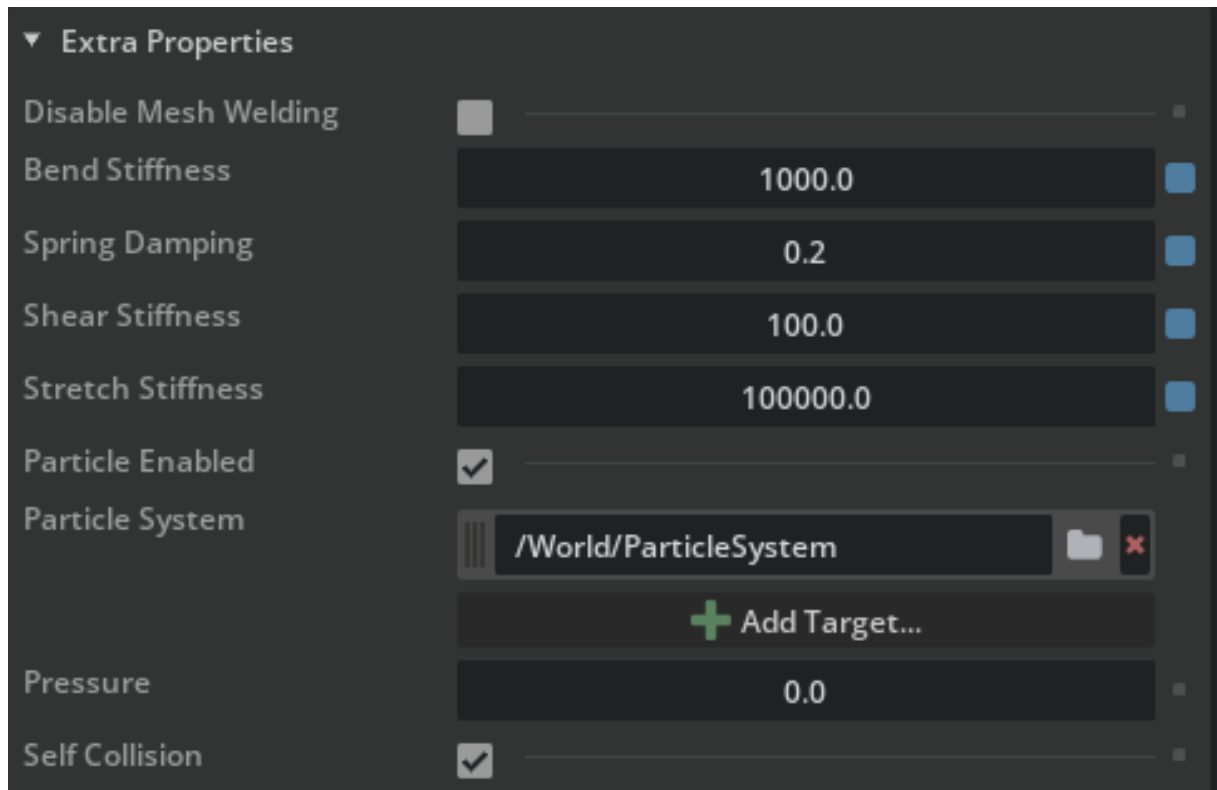
The image shows a dark-themed user interface for 'Extra Properties'. It contains a list of six material properties, each with a corresponding numerical value in a text input field. To the right of each input field is a small square icon. The 'Young's Modulus' icon is highlighted in blue.

▼ Extra Properties	
Damping Scale	1.0
Density	0.0
Dynamic Friction	0.25
Elasticity Damping	0.005
Poisson's Ratio	0.45
Young's Modulus	500.0

Fonte: Criada pelo autor.

ANEXO F – CÓDIGO DE EXEMPLO DE TECIDO NO OMNIVERSE

Figura 14 – Propriedades do Tecido no OMNIVERSE



Fonte: Criada pelo autor.